数论板子

生成时间: 2026-09-05

默认省略通用 C++ 头文件与空壳 main

目录

1 参考图表 3
1.1 表 3
1.2 图 4
2 基础与工具 5
2.1 数论/数论笔记部分/数论笔记(筛法) 5
2.2 数论/数论笔记部分/数论笔记(线性逆元) 8
2.3 数论/数论笔记部分/数论笔记(不定方程与同余方程组) 9
2.4 数论/Trick/数论小结论 12
3 卷积与反演 13
3.1 数论/数论笔记部分/数论笔记(狄利克雷卷积与莫比乌斯反演 1) 13
3.2 数论/数论笔记部分/数论笔记(狄利克雷卷积与莫比乌斯反演 2) 15
3.3 数论/数论笔记部分/数论笔记(炫酷反演魔术) 17
3.4 数论/数论笔记部分/数论笔记(和式变换) 29
4 组合与生成函数 31
4.1 数论/数论笔记部分/数论笔记(排列组合) 31
4.2 数论/数论笔记部分/数论笔记(排列组合进阶) 33
4.3 数论/数论笔记部分/数论笔记(生成函数) 39
5 变换与多项式 44
5.1 数论/数论笔记部分/FFT 笔记 44
5.2 数论/数论笔记部分/数论笔记(sosdp&fmt&fwt) 47
5.3 数论/数论笔记部分/数论笔记(阶,原根与 ntt) 51
6 Trick 与杂项 54
6.1 数论/Trick/数学相关 trick 54
6.2 数论/数论笔记部分/数论笔记(杂项) 54
6.3 数论/数论笔记部分/数学笔记(矩阵半环) 55

1 参考图表

1.1 表

n	$\log_{10} n$	$n!$	$C(n, n/2)$	$\mathrm{LCM}(1 \ldots n)$
2	0.30102999	2	2	2
3	0.47712125	6	3	6
4	0.60205999	24	6	12
5	0.69897000	120	10	60
6	0.77815125	720	20	60
7	0.84509804	5040	35	420
8	0.90308998	40320	70	840
9	0.95424251	362880	126	2520
10	1	3628800	252	2520
11	1.04139269	39916800	462	27720
12	1.07918125	479001600	924	27720
15	1.17609126	1.31e12	6435	360360
20	1.30103000	2.43e18	184756	232792560
25	1.39794001	1.55e25	5200300	26771144400
30	1.47712125	2.65e32	155117520	1.444e14

$n \leq$	10	100	1e3	1e4	1e5	1e6
$\max\{\omega(n)\}$	2	3	4	5	6	7
$\max\{d(n)\}$	4	12	32	64	128	240
$\pi(n)$	4	25	168	1229	9592	78498
$n \leq$	1e7	1e8	1e9	1e10	1e11	1e12
$\max\{\omega(n)\}$	8	8	9	10	10	11
$\max\{d(n)\}$	448	768	1344	2304	4032	6720
$\pi(n)$	664579	5761455	5.08e7	4.55e8	4.12e9	3.7e10
$n \leq$	1e13	1e14	1e15	1e16	1e17	1e18
$\max\{\omega(n)\}$	12	12	13	13	14	15
$\max\{d(n)\}$	10752	17280	26880	41472	64512	103680
$\pi(n)$	Prime number theorem: $\pi(x) \sim x/\log(x)$					

1.2 图

U 群常见问题速查

多项式复杂度

可以除 w 或除 $\log$，很难降次数。ω 为矩阵乘法复杂度指数下界。

logbit-OV、LCS、 两数最大 or/最小 and	$O(n^2)$
nbit-OV	$O(n^\omega)$
链 mex	$O(n^{\omega/2})$
$\min+$ 卷积	$O(n^2)$
3SUM、4SUM、两对数和相等、 三点共线、三线共点、 长为 3 等差数列	$O(n^2)$
3XOR	$O(n^2)$
APSP、$\min+$ 矩阵乘法	$O(n^3)$
$+\times$/01 矩阵乘/矩阵求逆/高斯消元	$O(n^\omega)$
$\min\max$ 矩阵乘	$O(n^{2\sim(\omega+3)/2})$
and or 矩阵乘、稠密图传递闭包	$O(n^{2\sim\omega})$
稀疏图传递闭包	$O(n^2)$
区间相等对/逆序对、链颜色数、 稀疏图三元环计数、行加列求和、 区间 ±1 全局数 0	$O(n^{\omega/2\sim2\omega/(\omega+1)})$

——From CommonAnts

图灵奖

参考资料：学会计算模型，复杂性，归约/问题类等计算理论基本思想后，自行搜索精细复杂性理论入门讲解 [Fine-grained complexity]

相关 OI 博客：

- lca 的讲解（bilibili 视频 @蔡德仁）
- lxl/crz 的相关讲解（资料集）
- EI《一些经典问题比暴力快一点点的算法》（博客）
- Ynoi《浅谈归约矩乘》（洛谷博客）
- Futari《归约矩乘》（洛谷博客）
- do-while-true《一小类矩阵乘法相关归约》（博客）
- hqztrue《浅谈矩阵乘法在算法竞赛中的应用》（知乎）
- 杨敏行《浅谈复杂度及其在解决问题方面的应用》
  闫陈效《计算理论与 OI 中的难解问题》（2024 集训队论文集）

——From CommonAnts

NPC/NP-Hard/#P-Hard

- Karp’s 21 NP-Complete Problems
- m 个大小均为 c 的背包能否装下给定物品
  （强 NPC，即不存在伪多项式算法）
- 简单环计数
- 二分图完美匹配计数
- 2-SAT 计数
- 拓扑序计数
- 无向图欧拉回路/欧拉路径计数

其他

- vuqa 是什么意思?
  van 能的 UOJ 群啊
- 那个画图的网站?
  csacademy.com/app/graph_editor/
- 那个查原题的网站?
  www.yuantiji.ac/zh
- 那个 AtCoder 评分的网站?
  kenkoooo.com/atcoder/#/table/
- 那个能看 Codeforces 题目整理的网站?
  cftracker.netlify.app/contests
- 那个看编译得到的汇编的网站?
  godbolt.org
- 那个把 C 语言类型转成人话的网站?
  cdecl.org
- XXX 用 LaTeX 怎么打?
  detexify.kirelabs.org/classify.html
  许多网站使用 KaTeX 渲染公式，相应文档见
  katex.org/docs/supported
- 怎么做 NOI 排版风格的题面?
  官方工具：TUACK + LaTeX
  广告：/Wallbreaker5th/fuzzy-cnoi-statement
  /Wallbreaker5th/OI-statement-LaTeX
- 那个 U 群常见问题速查的图?



Visit /Wallbreaker5th/vuqa for the source code and the latest version.

约数个数/不同质因子个数表

$n\le$	10^1	10^2	10^3	10^4	10^5	10^6	10^7	10^8	10^9
$\max\{\omega(n)\}$	2	3	4	5	6	7	8	8	9
$\max\{d(n)\}$	4	12	32	64	128	240	448	768	1344
$n\le$	10^{10}	10^{11}	10^{12}	10^{13}	10^{14}	10^{15}	10^{16}	10^{17}	10^{18}
$\max\{\omega(n)\}$	10	10	11	12	12	13	13	14	15
$\max\{d(n)\}$	2304	4032	6720	10752	17280	26880	41472	64512	103680

2 基础与工具

2.1 数论/数论笔记部分/数论笔记(筛法)

2.1.1 欧拉函数的定义

$1 \sim n$ 中与 n 互质的数的个数称为欧拉函数，记为 $\varphi(n)$

例：$\varphi(1)=1, \varphi(2)=1, \varphi(3)=2, \varphi(4)=2, \varphi(5)=4$

2.1.2 欧拉函数的性质

1. 若 p 是质数，则 $\varphi(p)=p-1$
2. 若 p 是质数，则 $\varphi\left(p^k\right)=(p-1)p^{k-1}$
3. 积性函数：若 $\gcd(m,n)=1$，则 $\varphi(mn)=\varphi(m)\varphi(n)$

2.1.3 欧拉函数的计算公式

由唯一分解定理 $n=\prod_{i=1}^{s} p_i^{\alpha_i}=p_1^{\alpha_1}p_2^{\alpha_2}\cdots p_s^{\alpha_s}$，

$$
\begin{aligned}
\varphi(n) &= \prod_{i=1}^{s}\varphi\left(p_i^{\alpha_i}\right) \\
&= \prod_{i=1}^{s} p_i^{\alpha_i-1}(p_i-1) \\
&= \prod_{i=1}^{s} p_i^{\alpha_i}\left(1-\frac{1}{p_i}\right) \\
&= \left(\prod_{i=1}^{s} p_i^{\alpha_i}\right)\times\left(\prod_{i=1}^{s}\left(1-\frac{1}{p_i}\right)\right) \\
&= n\times\prod_{i=1}^{s}\frac{p_i-1}{p_i} \\
&= n\times\frac{p_1-1}{p_1}\times\frac{p_2-1}{p_2}\times\cdots\times\frac{p_s-1}{p_s}
\end{aligned}
$$

欧拉函数可以写成 $n\prod_{p\mid n}\left(1-\frac{1}{p}\right)$；乘上的修正因子只和出现过的不同质因子有关，但整体值仍然含有 n，因此仍会随质因子的次数变化。

例：$\varphi(12)=12\times\frac{2-1}{2}\times\frac{3-1}{3}=4$

2.1.4 筛法求欧拉函数

若 i 是质数，$\varphi[i]=i-1$。

在线性筛中，每个合数 m 都是被最小的质因子筛掉的。 设 p_j 是 m 的最小质因子，则 m 通过 $m=p_j\times i$ 筛掉。

2.1.4.1 分两种情况计算

1. 若 i 能被 p_j 整除（即 $i\equiv 0\pmod{p_j}$），则 i 包含了 m 的所有质因子：

$$
\begin{aligned}
\varphi(m) &= m\times\prod_{k=1}^{s}\frac{p_k-1}{p_k} \\
&= p_j\times i\times\prod_{k=1}^{s}\frac{p_k-1}{p_k} \\
&= p_j\times\varphi(i)
\end{aligned}
$$

例：$\varphi(12)=\varphi(2\times 6)=2\times\varphi(6)$

2. 若 i 不能被 p_j 整除（即 $\gcd\left(i,p_j\right)=1$），则 i 和 p_j 互质：

$$
\begin{aligned}
\varphi(m) &= \varphi\left(p_j\times i\right) \\
&= \varphi\left(p_j\right)\times\varphi(i) \\
&= \left(p_j-1\right)\times\varphi(i)
\end{aligned}
$$

例：$\varphi(75)=\varphi(3\times 25)=(3-1)\times\varphi(25)$

```
vector<int> euler()
{
    vector<int> phi(n+1);
    phi[1]=1;
    vector<int> primes;
    vector<bool>v(n+1,0);
    for(int i=2;i<=n;i++)
    {
        if(!v[i])primes.push_back(i),phi[i]=i-1;
        for(int j=0;j<primes.size()&&primes[j]*i<=n;j++)
        {
            int m=primes[j]*i;
```

```
            v[m]=1;
            if(i%primes[j]==0)
            {
                phi[m]=phi[i]*primes[j];
                break;
            }
            else phi[m]=phi[i]*(primes[j]-1);
        }
    }
}
```

2.1.5 筛法求约数个数

2.1.5.1 问题

给定整数 n $(n \leq 10^6)$，输出 $1 \sim n$ 中每个数的约数个数。

2.1.5.2 约数个数定理

若正整数 n 有质因数分解 $n = \prod_{i=1}^{s} p_i^{\alpha_i}$，则约数个数为：

$$d(n) = \prod_{i=1}^{s} (\alpha_i + 1)$$

2.1.5.3 证明

- 对每个质因子 $p_i^{\alpha_i}$，其约数可取 $p_i^0, p_i^1, \cdots, p_i^{\alpha_i}$ 共 $(\alpha_i + 1)$ 种选择
- 根据乘法原理，总约数个数为各质因子选择数的乘积：

$$d(n) = (\alpha_1 + 1) \times (\alpha_2 + 1) \times \cdots \times (\alpha_s + 1)$$

2.1.5.4 筛法求约数个数

记 a[i]为 i 的最小质因子的次数，d[i]为 i 的约数个数。

若 i 是质数，

$$a[i] = 1, \quad d[i] = 2$$

在线性筛中，每个合数 m 都是被最小的质因子筛掉的。 设 p_j 是 m 的最小质因子，则 m 通过 $m = p_j \times i$ 筛掉。

(1) 若 i 能被 p_j 整除，则 p_j 一定是 i 的最小质因子。

$$a[m] = a[i] + 1;$$

$$d[i] = (a[i] + 1) \times \cdots, \quad d[m] = (a[m] + 1) \times \cdots$$

于是

$$d[m] = d[i] \times \frac{a[m] + 1}{a[i] + 1}$$

(2) 若 i 不能被 p_j 整除，则 i 不包含质因子 p_j。

$$a[m] = 1, \quad d[m] = d[i] \times (1 + 1)$$

```
//O(n)求1-n的约数个数
vector<int> d()
{
    vector<int> a(n+1),d(n+1);
    vector<int> primes;
    vector<bool>v(n+1,0);
    for(int i=2;i<=n;i++)
    {
        if(!v[i])
        {
            primes.push_back(i);
            a[i]=1,d[i]=2;
        }
        for(int j=0;j<primes.size()&&primes[j]*i<=n;j++)
        {
            int m=primes[j]*i;
            v[m]=1;
            if(i%primes[j]==0)
            {
                a[m]=a[i]+1;
                d[m]=d[i]/(a[i]+1)*(a[m]+1);
                break;
            }
            else
            {
                a[m]=1;
                d[m]=d[i]*2;
            }
        }
    }
}
```

2.1.6 约数和定理

若 $n = \prod_{i=1}^{s} p_i^{\alpha_i}$，则 $f(n) = \prod_{i=1}^{s} \sum_{j=0}^{\alpha_i} p_i^j$

证明：

$p_i^{\alpha_i}$ 的约数有 $p_i^0, p_i^1, \cdots, p_i^{\alpha_i}$ 共 (α_i+1) 个，其约数和为 $\sum_{j=0}^{\alpha_i} p_i^j$。

根据乘法原理，

$$f(n)=\prod_{i=1}^{s}\sum_{j=0}^{\alpha_i} p_i^j$$

例：

$$12=2^2\times 3^1,$$

$$f(12)=(1+2+4)\times(1+3)=7\times 4=28$$

2.1.7 筛法求约数和
记 g[i]为 i 的最小质因子的幂和 1 + p^1 +p^2 + ⋯ + p^k，f[i]为 i 的约数和。

若 i 是质数，

$$g[i]=f[i]=i+1$$

在线性筛中，每个合数 m 都是被最小的质因子筛掉的。设 p_j 是 m 的最小质因子，则 m 通过 $m=i\times p_j$ 筛掉。

(1) 若 i 能被 p_j 整除，则 p_j 一定也是 i 的最小质因子

$$g[i]=p_j^0+p_j^1+\cdots+p_j^{\alpha_j},\quad g[m]=p_j^0+p_j^1+\cdots+p_j^{\alpha_j+1}$$

$$f[i]=g[i]\times\cdots,\quad f[m]=g[m]\times\cdots$$

于是

$$f[m]=f[i]\times\frac{g[m]}{g[i]}$$

(2) 若 i 不能被 p_j 整除，则 i 不包含质因子 p_j。

$$g[m]=1+p_j$$

$$f[m]=g[m]\times f[i]$$

```
//O(n)求1-n的约数和
vector<int> sumd()
{
    vector<int> g(n+1),f(n+1);
    vector<int> primes;
    vector<bool>v(n+1,0);
    g[1]=f[1]=1;
    for(int i=2;i<=n;i++)
    {
        if(!v[i])
        {
            primes.push_back(i);
            f[i]=g[i]=i+1;
        }
        for(int j=0;j<primes.size()&&primes[j]*i<=n;j++)
        {
            int m=primes[j]*i;
            v[m]=1;
            if(i%primes[j]==0)
            {
                g[m]=g[i]*primes[j]+1;
                f[m]=f[i]*g[m]/g[i];
                break;
            }
            else
            {
                g[m]=primes[j]+1;
                f[m]=f[i]*g[m];
            }
        }
    }
}
```

2.1.8 唯一分解定理

$$n=\prod_{i=1}^{s} p_i^{\alpha_i}=p_1^{\alpha_1}p_2^{\alpha_2}\cdots p_s^{\alpha_s}$$

2.1.9 莫比乌斯函数定义
莫比乌斯函数记作 $\mu(n)$，它是一个经典的数论函数，定义如下：

- $\mu(1)=1$
- 如果 n 含有平方因子（即存在某个质数 p，使得 $p^2\mid n$），则 $\mu(n)=0$

- 如果 n 是 k 个互不相同的质数的乘积（即 $n = p_1 p_2 \cdots p_k$），则：

$$\mu(n) = (-1)^k$$

2.1.10 筛法求莫比乌斯函数
若 i 是质数，$\mu[i] = -1$。 在线性筛中，每个合数 m 都是被最小的质因子筛掉的。 设 p_j 是 m 的最小质因子，则 m 通过 $m = i \times p_j$ 筛掉。

(1) 若 i 能被 p_j 整除，则 i 也包含质因子 p_j。

$$\mu[m] = 0$$

(2) 若 i 不能被 p_j 整除，则 m 比 i 多一个不同的质因子 p_j

- 若 $\mu[i] = -1$, 则 $\mu[m] = 1$
- 若 $\mu[i] = 1$, 则 $\mu[m] = -1$
- 若 $\mu[i] = 0$, 则 $\mu[m] = 0$

综上，$\mu[m] = -\mu[i]$。

2.2 数论/数论笔记部分/数论笔记(线性逆元)

2.2.1 O(n) 求阶乘和阶乘逆元

2.2.1.1 推导目标

给定质数 p 和满足 $1 \le n < p$ 的 n，我们希望在线性时间内计算 1 到 n 的所有数在模 p 意义下的乘法逆元，即：

$$\text{求} \quad \forall 1 \le i \le n, \quad \text{使得} \quad r_i \cdot i \equiv 1 \pmod p \quad \text{的} \quad r_i$$

2.2.1.2 推导公式

我们设 $r_i = i^{-1} \bmod p$，有：

- $r_1 = 1$
- 对于 $1 < i \le n < p$，我们可以利用如下递推式求出 r_i：

$$r_i = \left(p - \left\lfloor \frac{p}{i} \right\rfloor\right) \cdot r_{p \bmod i} \bmod p$$

2.2.1.3 证明过程

考虑：

$$p = i \cdot \left\lfloor \frac{p}{i} \right\rfloor + (p \bmod i) \Rightarrow p \bmod i = p - i \cdot \left\lfloor \frac{p}{i} \right\rfloor$$

两边模 p:

$$i \cdot \left(\left\lfloor \frac{p}{i} \right\rfloor\right) \equiv -(p \bmod i) \pmod p$$

$$i \cdot \left(\left\lfloor \frac{p}{i} \right\rfloor\right) \cdot (p \bmod i)^{-1} \equiv -1 \pmod p$$

两边都乘上 -1：

$$i \cdot \left(-\left\lfloor \frac{p}{i} \right\rfloor\right) \cdot (p \bmod i)^{-1} \equiv 1 \pmod p$$

于是我们得出：

$$\boxed{\text{inv}\ [i] \equiv -\left(\left\lfloor \frac{p}{i} \right\rfloor\right) \cdot \text{inv}\ [p \bmod i] \pmod p}$$

再化简成无负数形式：

$$\boxed{\text{inv}\ [i] = (p - p/i) \cdot \text{inv}\ [p \% i] \bmod p}$$

这就是我们要用的递推式！

2.2.1.4 C++ 实现示例

```
void preC()
{
    inv[1]=1;
    for(int i=2;i<=n;i++)
    {
        inv[i]=(mod-mod/i)*inv[mod%i]%mod;
    }
    fac[0]=invfac[0]=1;
    for(int i=1;i<=n;i++)
    {
```

```
        fac[i]=fac[i-1]*i%mod;
        invfac[i]=invfac[i-1]*inv[i]%mod;
    }
}
```

2.2.1.5 时间复杂度

- 时间：$\mathcal{O}(n)$
- 空间：$\mathcal{O}(n)$
- 要求 p 是质数（否则不存在乘法逆元）

2.3 数论/数论笔记部分/数论笔记(不定方程与同余方程组)

2.3.1 前置芝士

exgcd 求解不定方程 ax+by=gcd(a,b) /线性同余方程 ax≡b(mod m) 的解

2.3.2 exgcd 求解不定方程 ax+by=gcd(a,b)

设

$$ax_1+by_1=gcd(a,b)$$

$$bx_2+(a\%b)y_2=gcd(b,a\%b)$$

由欧几里得定理可得

$$gcd(a,b)=gcd(b,a\%b)$$

于是

$$ax_1+by_1=bx_2+(a\%b)y_2$$

$$ax_1+by_1=bx_2+(a-\lfloor a/b\rfloor*b)y_2$$

整理

$$ax_1+by_1=ay_2+b(x_2-\lfloor a/b\rfloor*y_2)$$

于是

$$x_1=y_2$$

$$y_1=x_2-\lfloor a/b\rfloor*y_2$$

ax+by=gcd(a,b)即可通过最初的 x,y 求解

于是我们可以通过递归求解

```
int exgcd(int a,int b,int &x,int &y)//扩展欧几里得
{
    if(b==0)
    {
        //从x=1,y=0开始向上
        x=1;y=0;
        //ax+by=gcd(a,b)->ax=0->x=1,y=0
        return a;
    }
    //先求解gcd(a,b)
    int d=exgcd(b,a%b,x,y),t=x;
    x=y;y=t-a/b*y;
    return d;
}
```

2.3.3 exgcd 求解线性同余方程 ax≡b(mod m) 的解

$$ax\equiv b\pmod m$$

可写成

$$ax+mk=b$$

令

$$d=gcd(a,m)$$

于是我们先求解不定方程

$$ax+mk=d$$

若 $d\nmid b$ 则无解，否则得到解

$$x=x_0$$

$$k=k_0$$

于是我们得到原方程的解为

$$x_1 = x_0 * b/d$$

$$k_1 = k_0 * b/d$$

方程的任意解(对任意整数 t 成立)为

$$x = x_1 + \frac{m}{d}t$$

$$k = k_1 - \frac{a}{d}t$$

求最小的非负整数解

$$x = (x_1 \bmod t + t) \bmod t$$

其中

$$t = m/gcd(a, m)$$

如果要求最小正整数解，并且上式得到 $x = 0$，则取 $x = t$。

为何?

因为上述操作等价于

$$a/d * x \equiv b/d \pmod{m/d}$$

把 a/d 放到右边 会发现 x 是 模 m/d 的等价类

要用 exgcd 求解逆元的话，需要保证 gcd(a,m)=1

代入 exgcd(a,m,x,y)中,对 x 值域变换即可

其实就是

$$ax \equiv 1 \pmod{m}$$

可写成

$$ax + mk = gcd(a, m) = 1$$

罢了

2.3.4 中国剩余定理

求解同余方程组

$$\begin{cases} x \equiv a_1 \pmod{m_1} \\ x \equiv a_2 \pmod{m_2} \\ \cdots \\ x \equiv a_k \pmod{m_k} \end{cases}$$

其中

$$m_1, m_2, ..., m_k \text{两两互质}$$

过程:

求

$$M = m_1 * m_2 * ... * m_k$$

对每个 m_i 求

$$M_i = M/m_i$$

$$M_i * M_i^{-1} \equiv 1 \pmod{m_i}$$

$$c_i = M_i^{-1} * M_i$$

于是

$$x = \sum_{i=1}^{k} a_i * c_i \pmod{M}$$

很显然的证明，对任意一个方程组：

$$x \equiv \sum_{i=1}^{k} a_i * c_i \pmod{m_i}$$

$$x \equiv a_i * M_i * M_i^{-1} \pmod{m_i}$$

$$x \equiv a_i \pmod{m_i} * (M_i * M_i^{-1} \pmod{m_i})$$

按定义

$$x \equiv a_i \pmod{m_i}$$

代码：

```
int CRT()
{
    int mul=accumulate(m.begin(),m.end(),1LL,
    [](int a,int b){return a*b;}),ans=0;
    for(int i=0;i<n;i++)
    {
        int M=mul/m[i],b,y;
        exgcd(M,m[i],b,y);//求M的逆元
        ans=(ans+nums[i]*M%mul*b%mul+mul)%mul;
    }
    return (ans%mul+mul)%mul;
}
```

2.3.5 扩展中国剩余定理

求解同余方程组

$$\begin{cases} x \equiv a_1 \pmod{m_1} \\ x \equiv a_2 \pmod{m_2} \\ \cdots \\ x \equiv a_k \pmod{m_k} \end{cases}$$

其中

$$m_1, m_2, ..., m_k \text{不两两互质}$$

过程:

考虑合并两个同余方程

$$\begin{cases} x \equiv a_1 \pmod{m_1} \\ x \equiv a_2 \pmod{m_2} \end{cases}$$

可写成不定方程

$$\begin{cases} x = a_1 + k_1 * m_1 \\ x = a_2 + k_2 * m_2 \end{cases}$$

消去 x

$$a_1 + k_1 * m_1 = a_2 + k_2 * m_2$$

于是我们得到了一个不定方程

$$k_1 * m_1 - k_2 * m_2 = a_2 - a_1$$

可通过 exgcd 求解

$$K_1 * m_1 - K_2 * m_2 = gcd(m_1, m_2)$$

于是

$$k_1 = \frac{a_2 - a_1}{gcd(m_1, m_2)} * K_1$$

$$k_2 = \frac{a_2 - a_1}{gcd(m_1, m_2)} * K_2$$

得到 x 的一个解

$$x_0 = a_1 + k_1 * m_1 = a_1 + \frac{a_2 - a_1}{gcd(m_1, m_2)} * K_1 * m_1$$

窝们很显然可以构造 x 的通解

$$x = x_0 + t * lcm(m_1, m_2)$$

于是进行形式转化

$$x \equiv x_0 \pmod{lcm(m_1, m_2)}$$

于是我们得到了两个同余方程的合并

```
int _exCRT()
{
    int M=m[0],ans=nums[0];
    //M: 合并后的模数, ans:合并后的余数
    for(int i=1;i<n;i++)
    {
        //当前方程:
        //x≡nums[i] (mod \ m[i])
        //x≡ans (mod \ M)
        //不定方程 ax+by=gcd(a,b)
        int a=M,b=m[i];
        int c=((nums[i]-ans)%b+b)%b;
        int x,y;
        int gcd=exgcd(a,b,x,y);
        int bg=b/gcd;
        if(c%gcd!=0) return -1;//判断有无解
        x=(x%bg+bg)%bg;//对x值域变换变成正数
        x=(x*c/gcd%bg+bg)%bg;//对x值域变换
        ans+=x*M;
        M*=bg;//更新M=lcm(M,m[i])=m[i]*M/gcd(M,m[i])
        ans=(ans%M+M)%M;
    }
    return (ans%M+M)%M;
}
```

2.4 数论/Trick/数论小结论

$\gcd$ 的重要恒等式：$\gcd\left(x_1, x_2, ..., x_n\right) = \gcd\left(x_1, x_2 - x_1, x_3 - x_2, ..., x_n - x_{n-1}\right)$。

对于一个前缀 $\gcd$ 序列 $g_i = \gcd\left(a_1, ..., a_i\right)$，这个序列 $g_1, g_2, ..., g_n$ 的不同取值最多只有 $O(\log V)$ 个。 因为下降是log的

gcd(x,y)=x xor y -> gcd(x,y)=|x-y|

gcd(x,y)=x+y -> gcd(x,y)=max(x,y)。

gcd(k,n)=1->gcd(n-k,n)=1 simga [gcd(i,n)=1]i=phi(n)n/2 https://ac.nowcoder.com/acm/contest/view-submission?submissionId=79094134&returnHomeType=1&uid=719203876

原来 aj*2>ai 的时候 aj 一定不整除 ai https://ac.nowcoder.com/acm/contest/view-submission?submissionId=79086887&returnHomeType=1&uid=719203876

c|ab 当仅当 (c/gcd(a,c))|b 考虑 ab=kc 令 g=gcd(a,c),有 (a/g) b=k *(c/g) 此时 a/g 和 c/g 互质,其实含义就是(c/g)|b,反代得 (c/gcd(a,c))|b

自然数 k 次和：

$$\sum_{i=1}^{n} i^2 = \frac{n(n+1)(2n+1)}{6}$$

$$\sum_{i=1}^{n} i^3 = \left[\frac{n(n+1)}{2}\right]^2 = \frac{n^2(n+1)^2}{4}$$

Raney 引理：如果一个整数序列 $x_1, x_2, ..., x_m$ 的每一项都不超过 1，且总和为 $S > 0$，那么在这个序列的 m 个循环移位中，恰好有 S 个移位满足“所有非空前缀和都严格大于 0”。

$$\sum_{i=1}^{n} \left\lfloor \frac{n}{i} \right\rfloor = 2 \sum_{i=1}^{\lfloor \sqrt{n} \rfloor} \left\lfloor \frac{n}{i} \right\rfloor - \lfloor \sqrt{n} \rfloor^2$$

3 卷积与反演

3.1 数论/数论笔记部分/数论笔记(狄利克雷卷积与莫比乌斯反演 1)

3.1.1 狄利克雷生成函数

数列 $\langle a_1, a_2, a_3, \cdots \rangle$ 的狄利克雷生成函数定义为：

$$F(x) = \frac{a_1}{1^x} + \frac{a_2}{2^x} + \frac{a_3}{3^x} + \cdots = \sum_{n=1}^{\infty} \frac{a_n}{n^x}$$

3.1.1.1 乘法运算（Dirichlet 卷积）

两个狄利克雷生成函数相乘：

$$\begin{aligned}\left(\sum_{i=1}^{\infty} \frac{a_i}{i^x}\right)\left(\sum_{j=1}^{\infty} \frac{b_j}{j^x}\right) &= \sum_{i=1}^{\infty}\sum_{j=1}^{\infty} \frac{a_i b_j}{(ij)^x} \\ &= \sum_{n=1}^{\infty} \frac{1}{n^x} \sum_{d \mid n} a_d b_{n/d}\end{aligned}$$

所以 $\frac{1}{n^x}$ 项的系数就是：

$$\sum_{d \mid n} a_d b_{n/d}$$

也就是枚举 n 的约数 d，把 a_d 和 $b_{n/d}$ 配起来。

例如：

项	系数
$\frac{1}{4^x}$	$a_1 b_4 + a_2 b_2 + a_4 b_1$
$\frac{1}{6^x}$	$a_1 b_6 + a_2 b_3 + a_3 b_2 + a_6 b_1$

3.1.2 一点和式的小结论

3.1.2.1 欧拉函数

欧拉函数 $\varphi(n)$ 表示小于等于 n 且与 n 互质的正整数个数：

$$\varphi(n) = \sum_{i=1}^{n} [\gcd(i, n) = 1]$$

欧拉函数值表：

n	1	2	3	4	5	6	7	8	9	10	11	12
$\varphi(n)$	1	1	2	2	4	2	6	4	6	4	10	4

一个很常用的式子：

$$\sum_{d \mid n} \varphi(d) = n$$

验证一下：

$$\begin{aligned}\varphi(1) &= 1 \\ \varphi(1) + \varphi(2) &= 1 + 1 = 2 \\ \varphi(1) + \varphi(3) &= 1 + 2 = 3 \\ \varphi(1) + \varphi(2) + \varphi(4) &= 1 + 1 + 2 = 4 \\ \varphi(1) + \varphi(5) &= 1 + 4 = 5 \\ \varphi(1) + \varphi(2) + \varphi(3) + \varphi(6) &= 1 + 1 + 2 + 2 = 6\end{aligned}$$

证明思路：考虑以 n 为分母的真分数，也就是 $[0, 1)$ 里的这些数：

$$\frac{0}{n}, \frac{1}{n}, \frac{2}{n}, \cdots, \frac{n-1}{n}$$

把它们都化成最简分数，再按化简后的分母分组。

3.1.2.2 示例演示（$n = 12$）

所有真分数：

$$\frac{0}{12}, \frac{1}{12}, \frac{2}{12}, \frac{3}{12}, \frac{4}{12}, \frac{5}{12}, \frac{6}{12}, \frac{7}{12}, \frac{8}{12}, \frac{9}{12}, \frac{10}{12}, \frac{11}{12}$$

化简后分组：

分母 d	最简分数	个数 $\varphi(d)$
1	$\frac{0}{1}$	$\varphi(1)=1$
2	$\frac{1}{2}$	$\varphi(2)=1$
3	$\frac{1}{3},\frac{2}{3}$	$\varphi(3)=2$
4	$\frac{1}{4},\frac{3}{4}$	$\varphi(4)=2$
6	$\frac{1}{6},\frac{5}{6}$	$\varphi(6)=2$
12	$\frac{1}{12},\frac{5}{12},\frac{7}{12},\frac{11}{12}$	$\varphi(12)=4$

验证等式：

$$\sum_{d\mid 12}\varphi(d)=\varphi(1)+\varphi(2)+\varphi(3)+\varphi(4)+\varphi(6)+\varphi(12)=1+1+2+2+2+4=12$$

注意到窝们的证明思路天然满足定义。

一般性证明：

1. 考虑所有分母为 n 的真分数：

$$\frac{k}{n}\quad(0\le k<n)$$

   共有 n 个分数。

2. 将每个分数化简为最简形式 $\frac{a}{d}$，其中 $d\mid n$ 且 $\gcd(a,d)=1$。

3. 对 n 的每个约数 d 分组：

   - 分母为 d 的分数个数为 $\varphi(d)$。
   - 分子 a 满足 $1\le a\le d$ 且 $\gcd(a,d)=1$。

4. 总和为：

$$\sum_{d\mid n}\varphi(d)=n$$

   即得证。

3.1.3 莫比乌斯函数

莫比乌斯函数 $\mu(n)$ 定义如下：

$$\mu(n)=\begin{cases}1 & 若\ \ n=1\\ (-1)^s & 若\ \ n=p_1p_2\cdots p_s（无平方因子的整数）\\ 0 & 若\ \ n\ \ 包含平方因子\end{cases}$$

莫比乌斯函数值表：

n	1	2	3	4	5	6	7	8	9	10	11	12
$\mu(n)$	1	-1	-1	0	-1	1	-1	0	0	1	-1	0

核心性质：

$$\sum_{d\mid n}\mu(d)=[n=1]=\begin{cases}1 n=1\\ 0 n>1\end{cases}$$

验证一下：

$$\begin{aligned}
&n=1:\quad \mu(1)=1\\
&n=2:\quad \mu(1)+\mu(2)=1+(-1)=0\\
&n=3:\quad \mu(1)+\mu(3)=1+(-1)=0\\
&n=4:\quad \mu(1)+\mu(2)+\mu(4)=1+(-1)+0=0\\
&n=6:\quad \mu(1)+\mu(2)+\mu(3)+\mu(6)=1+(-1)+(-1)+1=0
\end{aligned}$$

3.1.3.1 证明（$n>1$ 时和为 0）

设：

$$n=p_1^{a_1}p_2^{a_2}\cdots p_s^{a_s}$$

定义 n' 为 n 的平方自由部分：

$$n' = p_1 p_2 \cdots p_s$$

含平方因子的约数对 μ 的贡献为 0，所以：

$$\sum_{d \mid n} \mu(d) = \sum_{d \mid n'} \mu(d)$$

考虑 n 的质因子集合 S，其大小为 s：

- $\mu(d) \neq 0$ 的 d 对应 S 的子集。
- d 的质因子个数为 k 时，$\mu(d) = (-1)^k$。
- 由二项式定理：

$$\sum_{d \mid n'} \mu(d) = \sum_{k=0}^{s} (-1)^k \binom{s}{k} = (1 + (-1))^s = 0$$

例如 $n = 6$，$6 = 2^1 \times 3^1$，$S = \{2, 3\}$：

$$\begin{aligned}
\mu(1) &= (-1)^0 = 1 \quad &(\text{取 } 0 \text{ 个质因子}) \\
\mu(2) &= (-1)^1 = -1 \quad &(\text{取质因子 } 2) \\
\mu(3) &= (-1)^1 = -1 \quad &(\text{取质因子 } 3) \\
\mu(6) &= (-1)^2 = 1 \quad &(\text{取质因子 } 2, 3)
\end{aligned}$$

和为 $1 + (-1) + (-1) + 1 = 0$。

3.2 数论/数论笔记部分/数论笔记(狄利克雷卷积与莫比乌斯反演 2)

3.2.1 狄利克雷卷积

3.2.1.1 定义

设 $f(n), g(n)$ 是两个积性函数，其狄利克雷卷积定义为：

$$(f * g)(n) = \sum_{d \mid n} f(d) g\left(\frac{n}{d}\right) = \sum_{d \mid n} f\left(\frac{n}{d}\right) g(d)$$

注意跟狄利克雷生成函数形式上的相似性 读作：f 卷 g

3.2.1.2 示例

$$(f * g)(4) = f(1)g(4) + f(2)g(2) + f(4)g(1)$$

3.2.1.3 运算规律

1. 交换律：$f * g = g * f$
2. 结合律：$(f * g) * h = f * (g * h)$
3. 分配律：$(f + g) * h = f * h + g * h$

3.2.1.4 常用函数

函数名称	符号表示	定义
元函数	$\epsilon(n)$	$[n = 1] = \begin{cases} 1 & n = 1 \\ 0 & n > 1 \end{cases}$
常数函数	$1(n)$	1
恒等函数	$id(n)$	n
欧拉函数	$\varphi(n)$	$< n$ 且与 n 互质的数的个数
莫比乌斯函数	$\mu(n)$	$\begin{cases} 1 & n = 1 \\ (-1)^k & n = p_1 p_2 \cdots p_k \\ 0 & n \text{含平方因子} \end{cases}$

注意符号的读法 μ 读作“缪”，φ 读作“phi”，ϵ 读作“一穆西隆”

3.2.1.5 常用卷积关系

简记形式

1. $\sum_{d \mid n} \mu(d) = [n = 1] \quad \Leftrightarrow \quad \mu * 1 = \epsilon$
2. $\sum_{d \mid n} \varphi(d) = n \quad \Leftrightarrow \quad \varphi * 1 = id$
3. $\sum_{d \mid n} \mu(d) \frac{n}{d} = \varphi(n) \quad \Leftrightarrow \quad \mu * id = \varphi$
4. $f * \epsilon = f$
5. $f * 1 \neq f$

注意莫比乌斯函数是常数函数的逆元

3.2.1.6 证明
3.2.1.7 1. $\mu*1=\epsilon$

$$(\mu*1)(n)=\sum_{d\mid n}\mu(d)\cdot 1\left(\frac{n}{d}\right)=\sum_{d\mid n}\mu(d)=[n=1]=\epsilon(n)$$

3.2.1.8 2. $\varphi*1=id$

$$(\varphi*1)(n)=\sum_{d\mid n}\varphi(d)\cdot 1\left(\frac{n}{d}\right)=\sum_{d\mid n}\varphi(d)=n=id(n)$$

3.2.1.9 3. $\mu*id=\varphi$

$$(\mu*id)(n)=\sum_{d\mid n}\mu(d)\cdot id\left(\frac{n}{d}\right)=\sum_{d\mid n}\mu(d)\cdot\frac{n}{d}=\varphi(n)$$

$$\mu*id=\mu*(\varphi*1)=(\mu*1)*\varphi=\epsilon*\varphi=\varphi$$

3.2.1.10 4. $f*\epsilon=f$

$$(f*\epsilon)(n)=\sum_{d\mid n}f(d)\cdot\epsilon\left(\frac{n}{d}\right)=\sum_{d\mid n}f(d)\cdot\left[\frac{n}{d}=1\right]=f(n)$$

3.2.1.11 5. $f*1\neq f$

$$(f*1)(n)=\sum_{d\mid n}f(d)\cdot 1\left(\frac{n}{d}\right)=\sum_{d\mid n}f(d)\neq f(n)$$

3.2.2 莫比乌斯反演
其实就是一下几个式子(条件式变成和式)

$$\sum_{d\mid n}\varphi(d)=n$$

$$\sum_{d\mid n}\mu(d)=[n=1]=\begin{cases}1n=1\\0n>1\end{cases}$$

把 n 换成 $\gcd(a,b)$

$$\sum_{d\mid gcd(a,b)}\mu(d)=[gcd(a,b)=1]\begin{cases}1gcd(a,b)=1\\0gcd(a,b)>1\end{cases}$$

3.2.3 莫比乌斯变换
3.2.3.1 基本公式
设 $f(n),g(n)$ 均为积性函数，则：

$$f(n)=\sum_{d\mid n}g(d)\Leftrightarrow g(n)=\sum_{d\mid n}\mu(d)f\left(\frac{n}{d}\right)$$

即

$$f=g*1\Leftrightarrow g=\mu*f$$

- $f(n)$ 称为 $g(n)$ 的莫比乌斯变换
- $g(n)$ 称为 $f(n)$ 的莫比乌斯逆变换

注意对于一些函数 $f(n)$，如果很难直接求出它的值，而容易求出其倍数和或约数和 $g(n)$，那么可以通过莫比乌斯反演简化运算，求得 $f(n)$ 的值。

3.2.3.2 证明方法一（卷积形式）
3.2.3.3 正向推导

$$\begin{aligned}f&=g*1\\\mu*f&=\mu*(g*1)\\&=g*(\mu*1)\\&=g*\epsilon\\&=g\end{aligned}$$

3.2.3.4 逆向推导

$$\begin{aligned}g&=\mu*f\\g*1&=(\mu*f)*1\\&=f*(\mu*1)\\&=f*\epsilon\\&=f\end{aligned}$$

3.2.3.5 证明方法二（双重求和）

$$\begin{aligned}\sum_{d\mid n}\mu(d)f\left(\frac{n}{d}\right)&=\sum_{d\mid n}\mu(d)\sum_{k\mid \frac{n}{d}}g(k)\\&=\sum_{d\mid n}\sum_{k\mid \frac{n}{d}}\mu(d)g(k)\\&=\sum_{k\mid n}\sum_{d\mid \frac{n}{k}}\mu(d)g(k)\\&=\sum_{k\mid n}g(k)\left(\sum_{d\mid \frac{n}{k}}\mu(d)\right)\\&=\sum_{k\mid n}g(k)\cdot\epsilon\left(\frac{n}{k}\right)\\&=g(n)\end{aligned}$$

3.3 数论/数论笔记部分/数论笔记(炫酷反演魔术)

从广义莫比乌斯反演的角度思考各类反演魔术

此笔记主要总结之前的二项式反演/莫比乌斯反演 然后补充斯特林反演/mix-max 容斥/容斥的相关技术

3.3.1 一切的基石：广义莫比乌斯反演

我们在数论中熟悉的 $\sum_{d\mid n}\mu(d)=[n=1]$ 其实只是冰山一角。所有的反演公式本质上都是在一个特定的偏序集(DAG)上求解线性方程组。

3.3.1.1 1.1 定义

设 P 是一个局部有限的偏序集。

Zeta 函数 (前缀和) $\zeta(x,y)$：表示 x 是否小于等于 y。

$$\zeta(x,y)=\begin{cases}1 & x\le y\\0 & \text{otherwise}\end{cases}$$

$\zeta(x,y)=1$ 等价于 DAG 上存在从 x 到 y 的路径。

我们现在知道

$$g(y)=\sum_{x\le y}\zeta(x,y)f(x)$$

我们现在要求解

$$f(y)=\sum_{x\le y}\mu(x,y)g(x)$$

于是我们有这个式子

Möbius 函数 (差分系数) $\mu(x,y)$：

$$\mu(x,y)=\begin{cases}1 & x=y\\-\sum_{x\le z<y}\mu(x,z) & x<y\\0 & \text{otherwise}\end{cases}$$

或者说

$$\sum_{y\preccurlyeq z\preccurlyeq x}\mu(z,x)=[x=y]=\delta(x,y)$$

你想求 x 到 y 的莫比乌斯函数值？你就把从 x 出发，到 y 之前所有中间节点 z 的 $\mu(x,z)$ 加起来，取个负号，就是 $\mu(x,y)$。

怎么推导呢?

定义两个双变量函数 A,B 的狄利克雷卷积（Dirichlet Convolution）为：

$$(A*B)(x,y)=\sum_{x\le z\le y}A(x,z)B(z,y)$$

这本质上就是矩阵乘法。如果你把 $A(x,z)$ 看作矩阵 A 的第 x 行第 z 列元素，把 $B(z,y)$ 看作矩阵 B 的第 z 行第 y 列元素，上述公式就是 $(AB)_{xy}$。

推导单位元 $\boldsymbol{\delta}$

我们想要求解 $f(y)$，这需要逆运算。在做逆运算之前，必须先知道在这个乘法规则下，什么东西相当于数字 1（保持原值不变）。

设单位元函数为 $\delta(x,y)$。对于任意函数 A，必须满足 $A*\delta = A$。即：

$$\sum_{x\le z\le y} A(x,z)\delta(z,y) = A(x,y)$$

观察这个等式：1. 当 $\boldsymbol{x = y}$ 时：式子变为 $A(x,x)\delta(x,x) = A(x,x)$。为了对任意 A 成立，必须有 $\boldsymbol{\delta(x,x) = 1}$。

2. 当 $\boldsymbol{x < y}$ 时：式子展开为 $A(x,x)\delta(x,y) + ... + A(x,y)\delta(y,y) = A(x,y)$。我们在上一步已经确定了 $\delta(y,y) = 1$，所以式子最后那一项 $A(x,y)\delta(y,y)$ 已经等于等号右边的 $A(x,y)$ 了。这意味着，前面所有的项加起来必须等于 0：

$$A(x,x)\delta(x,y) + A(x,x')\delta(x',y) + ... = 0$$

为了让它对任意 A 都恒成立（无论 A 取什么值），必须要求所有系数 $\boldsymbol{\delta(z,y) = 0}$（其中 $z < y$）。

结论 1：要在这种卷积下保持不变，必须使用函数：

$$\delta(x,y) = [x = y] = \begin{cases} 1 x = y \\ 0 x \neq y \end{cases}$$

形式化已知条件

已知条件是 $g(y) = \sum_{x\le y} f(x)$。引入常函数 $\zeta(x,y) = 1$（当 $x \le y$ 时）。原式可以写成卷积形式（视 f, g 为向量，A, B 为矩阵）：

$$g(y) = \sum_{x\le y} f(x)\zeta(x,y)$$

在代数层面上，这等价于：

$$g = f*\zeta$$

推导 $\boldsymbol{\mu}$

我们的目标是求 f。既然 $g = f*\zeta$，我们在两边同乘一个 ζ 的逆元（记作 μ）：

$$g*\mu = (f*\zeta)*\mu$$

利用结合律：

$$g*\mu = f*(\zeta*\mu)$$

为了消掉 ζ，我们要求 μ 满足：

$$\zeta*\mu = \delta$$

这样就有：

$$g*\mu = f*\delta = f$$

即 $\boldsymbol{f(y) = \sum_{x\le y} g(x)\mu(x,y)}$。

现在的问题核心变成了：如何找到这个 $\boldsymbol{\mu}$，使得 $\boldsymbol{\zeta*\mu = \delta}$？

根据 $\zeta*\mu = \delta$ 的定义，展开卷积公式：

$$\sum_{x\le z\le y} \zeta(x,z)\mu(z,y) = \delta(x,y)$$

因为对于 $x \le z$，$\zeta(x,z)$ 恒等于 1，所以式子简化为：

$$\sum_{x\le z\le y} \mu(z,y) = \delta(x,y)$$

这也是 μ 定义里是 $\mu(x,z)$ 还是 $\mu(z,y)$ 的区别。由于左逆元等于右逆元（因为我们定义的卷积的形式），我们通常习惯计算 $\mu*\zeta = \delta$，即：

$$\sum_{x\le z\le y} \mu(x,z)\zeta(z,y) = \delta(x,y)$$

简化后（因为 $\zeta(z,y) = 1$）：

$$\sum_{x\le z\le y} \mu(x,z) = \delta(x,y)$$

我们对区间 $[x,y]$ 的大小进行分类讨论：

1. 当 $\boldsymbol{x = y}$ 时（区间长度为 1）：

$$\mu(x,x)=\delta(x,x)$$

$$\boldsymbol{\mu(x,x)=1}$$

2. 当 $\boldsymbol{x<y}$ 时（区间长度大于 1）：

$$\sum_{x\le z\le y}\mu(x,z)=\delta(x,y)$$

因为 $x\neq y$，所以 $\delta(x,y)=0$。

$$\sum_{x\le z\le y}\mu(x,z)=0$$

我们将末项（$z=y$ 的项）分离出来：

$$\left(\sum_{x\le z<y}\mu(x,z)\right)+\mu(x,y)=0$$

移项得到 $\mu(x,y)$ 的递推式：

$$\mu(x,y)=-\sum_{x\le z<y}\mu(x,z)$$

$Q.E.D$

3.3.1.2 1.2 从矩阵视角看待这个反演
所有的反演公式，都可以抽象为两个数列（或函数） f 和 g 之间的线性关系：

$$g(y)=\sum_{x\preccurlyeq y}\zeta(x,y)f(x)$$

这写成矩阵形式就是：

$$\mathbf{g}=\mathbf{Z}\times\mathbf{f}$$

其中 $\mathbf{Z}$ 是一个变换矩阵。 所谓的“反演”，就是求逆矩阵 $\mathbf{M}=\mathbf{Z}^{-1}$：

$$\mathbf{f}=\mathbf{Z}^{-1}\times\mathbf{g}=\mathbf{M}\times\mathbf{g}$$

对应的形式就是

$$f(x)=\sum_{y\preccurlyeq x}\mu(y,x)g(y)$$

3.3.1.3 1.3. 通用反演定理
于是我们得到了这个很好看的式子

若有两个定义在偏序集 P 上的函数 f,g 满足“和”的关系：

$$g(x)=\sum_{y\le x}\zeta(y,x)f(y)$$

则可以通过 μ 函数反解出 f：

$$f(x)=\sum_{y\le x}\mu(y,x)g(y)$$

3.3.1.4 1.4. 这暗示我们啥东西?
我们通过定义一个矩阵乘法状物的卷积，把所有在偏序集上的反演公式都统一了起来。

其中 $\zeta*\mu=\delta$ 是一个核心方程，μ 是 ζ 的逆元。

3.3.2 容斥原理与子集/超集反演
3.3.2.1 2.1 容斥原理及其补集形式
有一个集合的集合 $A=\{S_i\}$，那么：

$$\left|\bigcup_{i=1}^{n}S_i\right|=\sum_{i=1}^{n}(-1)^{i-1}\sum_{\{a_k\},a_k<a_{k+1}}\left|\bigcap_{j=1}^{i}S_{a_j}\right|$$

这是显然的，要证也可以通过二项式定理证明

容斥原理的补集形式

$$\left|\bigcap_{i=1}^{n}S_i\right|=|\ U\ |-\left|\bigcup_{i=1}^{n}\overline{S_i}\right|$$

其实画个 venn 图就能看出来

3.3.2.2 2.2 子集/超集反演

$$f(S)=\sum_{T\subseteq S}g(T)\Leftrightarrow g(S)=\sum_{T\subseteq S}(-1)^{|S|-|T|}f(T)$$

$$f(S)=\sum_{S\subseteq T}g(T)\Leftrightarrow g(S)=\sum_{S\subseteq T}(-1)^{|T|-|S|}f(T)$$

证明：

引理 μ 具有积性，具体来说，若偏序集 $P=P_1\times P_2$，则 $\mu_P((x_1,x_2),(y_1,y_2))=\mu_{P_1}(x_1,y_1)\cdot\mu_{P_2}(x_2,y_2)$。

证：μ 函数的唯一定义特征是：对于任意区间 $[Start,End]$，所有子元素的 μ 值之和必须等于 1（当 $Start=End$ 时）值之和必须等于 0（当 $Start<End$ 时）只要能证明 $\mu_P(x)\cdot\mu_Q(y)$ 满足这个特征，那它就是 $\mu_{P\times Q}$。

设 $S=(x_1,y_1)$ 是起点，$E=(x_2,y_2)$ 是终点。我们需要计算区间 $[S,E]$ 里所有元素的“分量 μ 乘积”之和：

$$\text{Sum}=\sum_{S\le(u,v)\le E}\mu_P(x_1,u)\cdot\mu_Q(y_1,v)$$

因为积偏序集中，u 和 v 的选取是独立的（$x_1\le u\le x_2$ 且 $y_1\le v\le y_2$），我们可以把双重求和拆成两个括号相乘：

$$\text{Sum}=\left(\sum_{x_1\le u\le x_2}\mu_P(x_1,u)\right)\times\left(\sum_{y_1\le v\le y_2}\mu_Q(y_1,v)\right)$$

根据 μ 的定义，括号里的求和结果只能是 1 或 0：左括号 $=\delta(x_1,x_2)$（只有 $x_1=x_2$ 时是 1，否则是 0）右括号 $=\delta(y_1,y_2)$（只有 $y_1=y_2$ 时是 1，否则是 0）

情况 A：起点 = 终点即 $x_1=x_2$ 且 $y_1=y_2$。此时 $\text{Sum}=1\times1=1$。$\Longrightarrow$ 满足定义！

情况 B：起点 < 终点这意味着 x 和 y 中至少有一个不等（例如 $x_1<x_2$）。那么对应的那个括号求和就是 0。

$$\text{Sum}=0\times(...)=0\quad\text{或}\quad(...)\times0=0$$

$\Longrightarrow$ 满足定义！

$Q.E.D$

布尔格（所有子集构成的格）同构于 k 个 链（Chain）$L_2=\{0,1\}$ 的直积。即：$\mathcal{P}(\{1,...,k\})\cong L_2\times L_2\times...\times L_2$。（每个元素选或不选对应 0 或 1）。Möbius 函数具有积性：若偏序集 $P=P_1\times P_2$，则 $\mu_P((x_1,x_2),(y_1,y_2))=\mu_{P_1}(x_1,y_1)\cdot\mu_{P_2}(x_2,y_2)$。对于单个链 $0\le1$：$\mu(0,0)=1$ $\mu(0,1)=-1$（因为 $\mu(0,0)+\mu(0,1)=0$）

对于集合 $A\subseteq B$，设 $|B|-|A|=k$。这相当于在 k 个维度上从 0 变到了 1，而在其他维度上保持不变。因此：

$$\mu(A,B)=\underbrace{(-1)\times(-1)\times...\times(-1)}_{k\text{ 次}}=(-1)^k$$

超集反演同理，证明留作作业

3.3.3 二项式反演 (Binomial Inversion)
二项式反演是集合包含偏序集（即布尔格）上的反演在计数问题中的投影。它处理的是“恰好满足 k 个性质”与“至少满足 k 个性质”之间的转换。
3.3.3.1 3.1 形式一（对称形式）
这是最对称的一种形式，常用于证明其他等式。

$$f(n)=\sum_{i=0}^{n}(-1)^i\binom{n}{i}g(i)\Leftrightarrow g(n)=\sum_{i=0}^{n}(-1)^i\binom{n}{i}f(i)$$

3.3.3.2 3.2 形式二（常用形式/多退少补）
通常 $f(i)$ 表示“钦定选 i 个满足性质，其余随意的方案数”，$g(i)$ 表示“恰好 i 个满足性质的方案数”。

$$f(n)=\sum_{i=0}^{n}\binom{n}{i}g(i)\Leftrightarrow g(n)=\sum_{i=0}^{n}(-1)^{n-i}\binom{n}{i}f(i)$$

1. 恰好 → 至多型

设 g(m) 代表 恰好 满足条件 (= m) 的方案数（即答案） 设 f(m) 代表 至多 满足条件 (≤ m) 的方案数

先求出 f(m)，再反演求 g(m)

$$f(m)=\sum_{i=0}^{m}\binom{m}{i}g(i)\quad\Rightarrow\quad g(m)=\sum_{i=0}^{m}(-1)^{m-i}\binom{m}{i}f(i)$$

2. 恰好 → 至少型

设 g(m) 代表 恰好 满足条件 (= m) 的方案数（即答案） 设 f(m) 代表 至少 满足条件 (⩾ m) 的方案数

先求出 f(m)，再反演求 g(m)

$$f(m)=\sum_{i=m}^{n}\binom{i}{m}g(i)\quad\Rightarrow\quad g(m)=\sum_{i=m}^{n}(-1)^{i-m}\binom{i}{m}f(i)$$

我们证明一下形式二：

直接套用子集反演公式：

$$f(n)=\sum_{i=0}^{n}\binom{n}{i}g(i)\Leftrightarrow f(S)=\sum_{T\subseteq S}g(T)$$

$$g(S)=\sum_{T\subseteq S}(-1)^{|S|-|T|}f(T)$$

我们只关心大小，所以把大小为 i 的集合合并

$$g(n)=\sum_{i=0}^{n}(-1)^{n-i}\binom{n}{i}f(i)$$

3.3.4 莫比乌斯反演

$$f(n)=\sum_{d\mid n}g(d)\Leftrightarrow g(n)=\sum_{d\mid n}\mu(d)f\left(\frac{n}{d}\right)$$

证明：

我们取偏序集为 正整数整除格 $D=(\mathbb{Z}^+,|)$。 关系定义为：$a\le b\Leftrightarrow a\mid b$。

此时，我们需要证明：关联代数 $\boldsymbol{I(D)}$ 可以退化为狄利克雷代数。

3.3.4.1 核心引理：区间的结构同构

命题：对于任意 $d\mid n$，偏序区间 $[d,n]$ 与区间 $[1,n/d]$ 是序同构的。

> 序同构：两个偏序集 A 和 B 是序同构的，当且仅当存在双射 $\phi:A\to B$，使得 ϕ 和 ϕ^{-1} 都保持偏序关系。
>
> 其实就是两个偏序集的 dag 相同

证明：

1. 定义映射 $\phi:[d,n]\to[1,n/d]$，法则为 $\phi(z)=z/d$。
2. 双射性：
   - 若 $z\in[d,n]$，则 $d\mid z\mid n$，故 $1\mid(z/d)\mid(n/d)$，即 $\phi(z)\in[1,n/d]$。显然这是可逆的。
3. 保序性：
   - 对于任意 $a,b\in[d,n]$，我们需要证明 $a\mid b\Leftrightarrow\phi(a)\mid\phi(b)$。
   - 根据整除定义：$a\mid b\Leftrightarrow\exists k\in\mathbb{Z},b=ak$。
   - 等式两边同除以 d：$b/d=(a/d)k\Leftrightarrow\phi(b)=\phi(a)k$。
   - 这等价于 $\phi(a)\mid\phi(b)$。

结论：区间 $[d,n]$ 的偏序结构完全等同于 $[1,n/d]$。

由于 $\mu(x,y)$ 的值仅由区间 $[x,y]$ 的拓扑结构决定（由 μ 的递归定义可证，同构的偏序集具有相同的 μ 值）。

由核心引理可知，区间 $[d,n]$ 的结构仅依赖于商 n/d。 因此，我们可以定义约化函数。

对于关联代数中的任意函数 $h(x,y)$，若其满足 $h(x,y)=h(kx,ky)$（即具有平移不变性），我们可以定义对应的一元算术函数 $\tilde{h}$：

$$\tilde{h}(k)\equiv h(1,k)$$

从而有：

$$h(d,n)=\tilde{h}\left(\frac{n}{d}\right)$$

检查 $\boldsymbol{\zeta}$ 和 $\boldsymbol{\mu}$ 是否符合此条件：

1. Zeta: $\zeta(d,n)=1\Leftrightarrow d\mid n$。 $\zeta(1,n/d)=1\Leftrightarrow 1\mid(n/d)$。 显然恒成立，故 $\zeta(d,n)=\tilde{\zeta}(n/d)=1$。 这里 $\tilde{\zeta}$ 就是数论中的常数函数 $\mathbf{1}(n)=1$。
2. Mu: 由于 μ 是 ζ 的逆元，且 ζ 具有平移不变性，由代数性质可知 μ 也必然继承此性质（同构区间的逆元结构也相同）。 定义数论莫比乌斯函数 $\mu_{NT}(k)=\mu(1,k)$。 则 $\mu(d,n)=\mu_{NT}(n/d)$。

现在我们将广义反演公式代入整除格环境。

广义形式：

$$f(n)=\sum_{d|n}g(d)\mu(d,n)$$

(注：这里 $d\le n$ 即 $d\mid n$)

代入降维后的 $\boldsymbol{\mu}$：

$$f(n)=\sum_{d|n}g(d)\mu_{NT}\left(\frac{n}{d}\right)$$

为了验证这个 μ_{NT} 确实是我们熟知的那个函数，我们检查它的一元卷积性质。在广义代数中：$\mu*\zeta=\delta$。对应的一元卷积（狄利克雷卷积）为：

$$\left(\tilde{\mu}*\tilde{\zeta}\right)(n)=\sum_{d|n}\tilde{\mu}(d)\tilde{\zeta}\left(\frac{n}{d}\right)=\sum_{d|n}\mu_{NT}(d)\cdot 1$$

而在广义代数中，$\delta(x,y)$ 降维后对应单位函数 $\epsilon(n)=[n=1]$。

为啥？此时$d=n,n/d=1$

因此：

$$\sum_{d|n}\mu_{NT}(d)=[n=1]$$

这正是数论中莫比乌斯函数的定义式。

3.3.4.2 最终结论

通过整除格区间 $[\boldsymbol{d},\boldsymbol{n}]\cong[\boldsymbol{1},\boldsymbol{n/d}]$ 的序同构，我们将定义在 $I(D)$ 上的广义莫比乌斯反演：

$$f(n)=\sum_{d|n}g(d)\mu(d,n)$$

严格推导为数论形式：

$$f(n)=\sum_{d|n}g(d)\mu_{NT}\left(\frac{n}{d}\right)$$

证毕。

以下是常用式子

$$\sum_{d\mid n}\varphi(d)=n$$

$$\sum_{d\mid n}\mu(d)=[n=1]=\begin{cases}1n=1\\0n>1\end{cases}$$

把 n 换成 $\gcd(a,b)$

$$\sum_{d\mid gcd(a,b)}\mu(d)=[gcd(a,b)=1]\begin{cases}1gcd(a,b)=1\\0gcd(a,b)>1\end{cases}$$

初等证明前已述及

3.3.5 Min-Max 容斥

Min-Max 容斥用于将“全集中最大值（最后出现）的问题”转化为“子集中最小值（最先出现）的问题”。在概率期望中，$\min$ 通常比 $\max$ 好算得多。

3.3.5.1 公式

对于全序集合 S 和其上的值 $\{x_i\}$：

$$\max_{i\in S}{}_k x_i=\sum_{T\subseteq S,T\neq\emptyset}(-1)^{|T|-k}\binom{|T|-1}{k-1}\min_{j\in T}x_j$$

$$\min_{i\in S}{}_k x_i=\sum_{T\subseteq S,T\neq\emptyset}(-1)^{|T|-k}\binom{|T|-1}{k-1}\max_{j\in T}x_j$$

其实 min_max 容斥 在期望意义下也满足：

$$E\left(\max_{i\in S}{}_k x_i\right)=\sum_{T\subseteq S,T\neq\emptyset}(-1)^{|T|-k}\binom{|T|-1}{k-1}E\left(\min_{j\in T}x_j\right)$$

$$E\left(\min_{i\in S} k x_i\right)=\sum_{T\subseteq S,T\neq\emptyset}(-1)^{|T|-k}\binom{|T|-1}{k-1}E\left(\max_{j\in T} x_j\right)$$

由期望线性性可知。

期望的线性性告诉我们：和的期望等于期望的和。 即 $E[\sum c_i\cdot V_i]=\sum c_i\cdot E[V_i]$，无论这些随机变量 V_i 之间是否独立。

$$\begin{aligned}E[Y]&=E\left[\sum_{T\subseteq S}a(|T|)\cdot Z_T\right]\\&=\sum_{T\subseteq S}a(|T|)\cdot E[Z_T]\quad(\text{常数提出，求和拆开})\end{aligned}$$

特殊情况 ($k=1$, 最大值):

$$\max(S)=\sum_{T\subseteq S,T\neq\emptyset}(-1)^{|T|-1}\min(T)$$

3.3.5.2 初等证明
我们需要构造一个关于集合大小的系数函数 $a(|T|)$，使得等式成立：

$$\text{第 }\ k\ \text{ 大值}=\sum_{T\subseteq S}a(|T|)\min(T)$$

1.分析元素的贡献 考虑 S 中第 r 大的那个元素，记为 $x_{(r)}$。 在右边的求和式 $\sum a(|T|)\min(T)$ 中，$x_{(r)}$ 何时会被计入？ 只有当 $x_{(r)}$ 是集合 T 的最小值时，$\min(T)$ 才会取到 $x_{(r)}$。

如果 $x_{(r)}$ 是 T 的最小值，那么 T 中的其他元素必须都比 $x_{(r)}$ 大。 在 S 中，比 $x_{(r)}$ 大的元素一共有 $r-1$ 个。 假设我们构造的集合 T 大小为 m。除去 $x_{(r)}$ 本身，我们需要从那 $r-1$ 个比它大的数中选出 $m-1$ 个。 方案数为 $\binom{r-1}{m-1}$。

所以，第 r 大的数 $x_{(r)}$ 在右边的总贡献系数是：

$$C(r)=\sum_m\binom{r-1}{m-1}a(m)$$

(这里枚举的是集合大小 m)

2.建立目标方程 我们希望左边计算出来的结果恰好是第 k 大的值。 这意味着：

当 $r=k$ 时（即它是第 k 大），总系数 $C(r)=1$。

当 $r\neq k$ 时，总系数 $C(r)=0$。

令 $N=r-1$ (表示比当前数大的数的个数)，目标方程为：

$$\sum_m\binom{N}{m-1}a(m)=[N=k-1]$$

令 $j=m-1$，则 $a(m)=a(j+1)$。方程化为：

$$\sum_{j=0}^{N}\binom{N}{j}a(j+1)=[N=k-1]$$

3.使用二项式反演求解系数 观察上式，形如 $F(N)=\sum\binom{N}{j}G(j)$。 其中 $F(N)=[N=k-1]$ 是已知结果，$G(j)=a(j+1)$ 是待求系数。 根据二项式反演形式二：

$$G(N)=\sum_{j=0}^{N}(-1)^{N-j}\binom{N}{j}F(j)$$

代入 $F(j)=[j=k-1]$：

$$a(N+1)=\sum_{j=0}^{N}(-1)^{N-j}\binom{N}{j}[j=k-1]$$

由于求和式中只有 $j=k-1$ 这一项非零（前提是 $N\geq k-1$），直接提取该项：

$$a(N+1)=(-1)^{N-(k-1)}\binom{N}{k-1}$$

4.还原变量 代入上式：

$$
\begin{aligned}
a(|\ T\ |) &= (-1)^{(|\ T\ |-1)-(k-1)}\binom{|\ T\ |-1}{k-1} \\
&= (-1)^{|\ T\ |-k}\binom{|\ T\ |-1}{k-1}
\end{aligned}
$$

max 和 min 在全序上是对称的,所以第二个式子就不证了

从广义莫反证明 min-max 容斥是有点难度的，留作读者证明

3.3.6 斯特林反演

斯特林反演揭示了普通幂与下降幂/上升幂之间的基底变换关系。

第二类斯特林数 (子集数) $\left\{ {n \atop k} \right\}$：将 n 个不同元素放入 k 个无区别盒子（非空）的方案数。

第一类斯特林数 (轮换数) $\left[{n \atop k} \right]$：将 n 个不同元素排成 k 个轮换（非空）的方案数。

斯特林反演

$$
f(n)=\sum_{i=0}^{n}\left\{ {n \atop i} \right\}g(i) \Leftrightarrow g(n)=\sum_{i=0}^{n}(-1)^{n-i}\left[{n \atop i} \right]f(i)
$$

3.3.6.1 前置知识

上升幂和下降幂

上升幂 $x^{\overline{n}}=\prod_{i=0}^{n-1}(x+i)$ 下降幂 $x^{\underline{n}}=\prod_{i=0}^{n-1}(x-i)=n!$

上升幂转普通幂

$$
x^{\overline{n}}=\sum_{i=0}^{n}\left[{n \atop i} \right]x^i
$$

证明： 考虑归纳证明： $n=0$ 时，等式两边皆为 1，等式成立 $n=1$ 时，等式左边等于 x，等式右边等于 x，等式成立。

$$
\begin{aligned}
x^{\overline{n+1}} &= (x+n)x^{\overline{n}} \\
&= (x+n)\sum_{i=0}^{n}\left[{n \atop i} \right]x^i \\
&= x\sum_{i=0}^{n}\left[{n \atop i} \right]x^i + n\sum_{i=0}^{n}\left[{n \atop i} \right]x^i \\
&= \sum_{i=1}^{n+1}\left[{n \atop i-1} \right]x^i + n\sum_{i=0}^{n+1}\left[{n \atop i} \right]x^i \\
&= \sum_{i=0}^{n+1}\left(\left[{n \atop i-1} \right] + n\left[{n \atop i} \right]\right)x^i \\
&= \sum_{i=0}^{n+1}\left[{n+1 \atop i} \right]x^i
\end{aligned}
$$

下降幂转普通幂

$$
x^{\underline{n}}=\sum_{i=0}^{n}\left[{n \atop i} \right](-1)^{n-i}x^i
$$

证明： 同样考虑归纳证明： $n=0$ 时，等式两边皆为 1，等式成立 $n=1$ 时，等式两边皆为 x，等式成立

$$
\begin{aligned}
x^{\underline{n+1}} &= (x-n)x^{\underline{n}} \\
&= (x-n)\sum_{i=0}^{n}\begin{bmatrix}n\\i\end{bmatrix}(-1)^{n-i}x^i \\
&= x\sum_{i=0}^{n}\begin{bmatrix}n\\i\end{bmatrix}(-1)^{n-i}x^i - n\sum_{i=0}^{n}\begin{bmatrix}n\\i\end{bmatrix}(-1)^{n-i}x^i \\
&= \sum_{i=0}^{n}\begin{bmatrix}n\\i\end{bmatrix}(-1)^{n-i}x^{i+1} - n\sum_{i=0}^{n}\begin{bmatrix}n\\i\end{bmatrix}(-1)^{n-i}x^i \\
&= \sum_{i=1}^{n+1}\begin{bmatrix}n\\i-1\end{bmatrix}(-1)^{n-i+1}x^i - n\sum_{i=0}^{n+1}\begin{bmatrix}n\\i\end{bmatrix}(-1)^{n-i}x^i \\
&= \sum_{i=1}^{n+1}\begin{bmatrix}n\\i-1\end{bmatrix}(-1)^{n-i+1}x^i + n\sum_{i=0}^{n+1}\begin{bmatrix}n\\i\end{bmatrix}(-1)^{n-i+1}x^i \\
&= \sum_{i=0}^{n+1}\left(\begin{bmatrix}n\\i-1\end{bmatrix} + n\begin{bmatrix}n\\i\end{bmatrix}\right)(-1)^{n-i+1}x^i \\
&= \sum_{i=0}^{n+1}\begin{bmatrix}n+1\\i\end{bmatrix}(-1)^{(n+1)-i}x^i
\end{aligned}
$$

普通幂转为下降幂

$$
x^n = \sum_{i=0}^{n}\begin{Bmatrix}n\\i\end{Bmatrix}x^{\underline{i}}
$$

证明：考虑组合意义 假设我们有：n 个不同的球（标号 $1,2,...,n$）x 个不同的盒子（标号 $1,2,...,x$） 我们要计算：把这 n 个球任意放入这 x 个盒子中，允许盒子为空，一共有多少种方案？左边 对于每一个球，我们都可以从 x 个盒子中任选一个放入。所以总方案数为 x^n。 右边 假设最后放完球后，恰好有 i 个盒子 里有球（即非空盒子数为 i） 1.选出这 i 个盒子我们在 x 个不同的盒子中，选出 i 个将被使用的盒子。方案数为组合数：$\binom{x}{i}$ 2.把 n 个球分给这 i 个盒子（暂时不区分盒子的顺序） 这相当于把 n 个不同的元素，划分成 i 个非空的集合（堆）。这正是第二类斯特林数 $\begin{Bmatrix}n\\i\end{Bmatrix}$ 的定义！ 3.把这 i 堆球放入刚才选好的 i 个盒子里因为盒子是不同的，我们需要决定哪一堆球放进哪一个盒子。这相当于 i 个元素的全排列。方案数为：$i!$ 方案总数为：

$$
\binom{x}{i} \times \begin{Bmatrix}n\\i\end{Bmatrix} \times i! = \begin{Bmatrix}n\\i\end{Bmatrix}x^{\underline{i}}
$$

普通幂转上升幂

$$
x^n = \sum_{i=0}^{n}\begin{Bmatrix}n\\i\end{Bmatrix}(-1)^{n-i}x^{\overline{i}}
$$

我们会发现一个关于负数的重要性质：

$$
(-x)^{\underline{n}} = (-1)^n x^{\overline{n}}
$$

推导引理：

$$
\begin{aligned}
(-x)^{\underline{n}} &= (-x)(-x-1)(-x-2)...(-x-n+1) \\
&= [(-1)(x)]\cdot[(-1)(x+1)]\cdot[(-1)(x+2)]...[(-1)(x+n-1)] \\
&= (-1)^n \cdot \underbrace{x(x+1)(x+2)...(x+n-1)}_{x^{\overline{n}}} \\
&= (-1)^n x^{\overline{n}}
\end{aligned}
$$

我们已知普通幂转下降幂的公式：

$$
x^n = \sum_{i=0}^{n}\begin{Bmatrix}n\\i\end{Bmatrix}x^{\underline{i}}
$$

在这个恒等式中，将 x 替换为 $-x$：

$$
(-x)^n = \sum_{i=0}^{n}\begin{Bmatrix}n\\i\end{Bmatrix}(-x)^{\underline{i}}
$$

左边展开为 $(-1)^n x^n$，右边利用引理代换：

$$
(-1)^n x^n = \sum_{i=0}^{n}\begin{Bmatrix}n\\i\end{Bmatrix}\left[(-1)^i x^{\overline{i}}\right]
$$

两边同时除以 $(-1)^n$（或者乘以 $(-1)^n$）：

$$\begin{aligned}x^n&=\frac{1}{(-1)^n}\sum_{i=0}^{n}\left\{{n\atop i}\right\}(-1)^i x^{\overline{i}}\\&=\sum_{i=0}^{n}\left\{{n\atop i}\right\}\frac{(-1)^i}{(-1)^n}x^{\overline{i}}\\&=\sum_{i=0}^{n}\left\{{n\atop i}\right\}(-1)^{i-n}x^{\overline{i}}\end{aligned}$$

由于 $(-1)^{i-n}=(-1)^{n-i}$（指数差偶数倍不影响符号），整理得：

$$x^n=\sum_{i=0}^{n}\left\{{n\atop i}\right\}(-1)^{n-i}x^{\overline{i}}$$

反转公式

$$\sum_{i=m}^{n}(-1)^{n-i}\left\{{n\atop i}\right\}\left[{i\atop m}\right]=[m=n]$$

$$\sum_{i=m}^{n}(-1)^{m-i}\left[{n\atop i}\right]\left\{{i\atop m}\right\}=[m=n]$$

反转公式（1）证明(利用下降幂转普通幂/普通幂转下降幂)：

$$\begin{aligned}x^{\underline{n}}&=\sum_{i=0}^{n}\left[{n\atop i}\right](-1)^{n-i}x^i\\&=\sum_{i=0}^{n}\left[{n\atop i}\right](-1)^{n-i}\sum_{j=0}^{i}\left\{{i\atop j}\right\}x^{\underline{j}}\\&=\sum_{i=0}^{n}x^{\underline{i}}\sum_{j=i}^{n}(-1)^{n-j}\left[{n\atop j}\right]\left\{{j\atop i}\right\}\end{aligned}$$

当 $i=n$ 时，等式右边为 $x^{\underline{n}}$，等式成立 当 $i<n$ 时，等式右边为 要为0，等式成立

反转公式（2）证明：

$$\begin{aligned}x^n&=\sum_{i=0}^{n}\left\{{n\atop i}\right\}x^{\underline{i}}\\&=\sum_{i=0}^{n}\left\{{n\atop i}\right\}(-1)^i(-x)^{\overline{i}}\\&=\sum_{i=0}^{n}\left\{{n\atop i}\right\}(-1)^i\sum_{j=0}^{i}\left[{i\atop j}\right](-x)^j\\&=\sum_{i=0}^{n}x^i\sum_{j=i}^{n}(-1)^{i-j}\left\{{n\atop j}\right\}\left[{j\atop i}\right]\end{aligned}$$

同理

斯特林反演

$$f(n)=\sum_{i=0}^{n}\left\{{n\atop i}\right\}g(i)\Leftrightarrow g(n)=\sum_{i=0}^{n}(-1)^{n-i}\left[{n\atop i}\right]f(i)$$

上式中一二类斯特林数可以互换位置。

证明： 已知： $g(n)=\sum_{i=0}^{n}(-1)^{n-i}\left[{n\atop i}\right]f(i)$

$$\begin{aligned}f(n)&=\sum_{i=0}[i=n]f(i)\\&=\sum_{i=0}^{n}\sum_{j=i}^{n}\left\{{n\atop j}\right\}\left[{j\atop i}\right](-1)^{j-i}f(i)\\&=\sum_{i=0}^{n}\left\{{n\atop i}\right\}\sum_{j=0}^{i}(-1)^{i-j}\left[{i\atop j}\right]f(j)\\&=\sum_{i=0}^{n}\left\{{n\atop i}\right\}g(i)\end{aligned}$$

3.3.6.2 另一个视角
斯特林反演本质上是“集合分划格”（Partition Lattice）上的广义莫比乌斯反演。

就像：* 二项式反演 对应 子集格（布尔格） 上的莫比乌斯反演。* 数论莫比乌斯反演 对应 整除格 上的莫比乌斯反演。* 斯特林反演 对应 集合分划格 上的莫比乌斯反演。

3.3.6.2.1 1. 什么是集合分划格 Π_n？

我们定义一个偏序集（Poset）$P = \Pi_n$。* 元素：集合 $S = \{1, 2, ..., n\}$ 的所有分划。* 例如 $n = 3$，$\{\{1, 2\}, \{3\}\}$ 就是一个分划。* 偏序关系 $\leq$：* 对于两个分划 σ 和 τ，如果 σ 中的每一个块（block）都完全包含在 τ 的某一个块中，我们称 $\sigma \leq \tau$（σ 比 τ 更“细”，或者 τ 比 σ 更“粗”）。* 最小元 $\hat{\mathbf{0}}$：$\{\{1\}, \{2\}, ..., \{n\}\}$（全部分开，最细）。* 最大元 $\hat{\mathbf{1}}$：$\{\{1, 2, ..., n\}\}$（全部在一起，最粗）。

3.3.6.2.2 2. 这个格上的 Zeta 和 Möbius 函数

在 Π_n 上定义广义莫比乌斯反演的两个核心函数：

Zeta 函数 $\zeta(\sigma, \tau)$ 定义为：若 $\sigma \leq \tau$，则为 1，否则为 0。它对应着求和操作，也就是我们熟悉的“第二类斯特林数”方向。

Möbius 函数 $\mu(\sigma, \tau)$ 它是 ζ 的逆。在集合分划格中，μ 的取值非常特殊：对于区间 $[\hat{0}, \hat{1}]$（即从全分散到全聚合），有著名的定理：

$$\mu\big(\hat{0}, \hat{1}\big) = (-1)^{n-1}(n-1)!$$

咋证？我们要证的目标是 $M_n = (-1)^{n-1}(n-1)!$，其中 M_n 简记为 n 个元素分划格最顶端的 $\mu\big(\hat{0}, \hat{1}\big)$。利用 $\sum \mu = 0$ 构造递推 1. 莫比乌斯函数的定义 对于任意 $n > 1$，分划格 Π_n 中所有元素的 μ 值之和必须为 0：

$$\sum_{\sigma \in \Pi_n} \mu\big(\hat{0}, \sigma\big) = 0$$

换句话说：

$$M_n = -\sum_{\sigma \neq \hat{1}} \mu\big(\hat{0}, \sigma\big)$$

(最顶上的那一项，等于底下所有项之和的相反数) 2. 巧妙的分组 我们根据“元素 $\boldsymbol{n}$ 所在的块的大小”来对所有的分划 σ 进行归类。设元素 n 所在的块为 B，设 $|B| = i$。i 的取值范围是 1 到 n。选出这个块 B 的方案数是 $\binom{n-1}{i-1}$（因为 n 必须在里面，从剩下 $n-1$ 个里选 $i-1$ 个陪它）。3. 结构的乘积 对于一个固定的块 B（大小为 i），剩下的 $n-i$ 个元素组成了某种分划 τ。因为块之间互不干扰，μ 值具有乘积性质：

$$\mu\big(\hat{0}, \sigma\big) = \underbrace{M_i}_{\text{块 } B \text{ 的贡献}} \times \underbrace{\mu(\text{剩余部分的 } \tau)}_{\text{剩下的贡献}}$$

4. 所有的项求和 把刚才的 $\sum \mu = 0$ 按照 i 展开：

$$0 = \sum_{i=1}^{n} \binom{n-1}{i-1} M_i \times \underbrace{\left(\sum_{\tau \in \Pi_{n-i}} \mu\big(\hat{0}, \tau\big) \right)}_{\text{剩余部分的所有情况之和}}$$

5. 奇迹发生了 请盯着最后那个括号里的部分：$\sum_{\tau \in \Pi_{n-i}} \mu\big(\hat{0}, \tau\big)$。这是一个大小为 $n-i$ 的全部分划格的 μ 之和。根据莫比乌斯函数的定义，只要 $n - i > 1$，这个和就是 0！这个和不为 0 的只有两种情况：$\boldsymbol{n - i = 1}$：只剩 1 个元素，和为 $M_1 = 1$。$\boldsymbol{n - i = 0}$：没有剩余元素（即 $i = n$），定义为 1。6. 最后的递推 因为只有 $n - i = 0$ 和 $n - i = 1$ 有值，长长的求和式瞬间只剩下两项：当 $\boldsymbol{i = n}$ 时（全在一起）：系数是 $\binom{n-1}{n-1} M_n \times 1 = M_n$ 当 $\boldsymbol{i = n - 1}$ 时（剩一个孤立点）：系数是 $\binom{n-1}{n-2} M_{n-1} \times M_1 = (n-1)M_{n-1}$ 方程变成了：

$$M_n + (n-1)M_{n-1} = 0$$

$$\Rightarrow M_n = -(n-1)M_{n-1}$$

7. 求解 $M_1 = 1$ $M_2 = -1 \cdot M_1 = -1$ $M_3 = -2 \cdot M_2 = 2 \cdots M_n = (-1)^{n-1}(n-1)!$

更一般地，对于任意 $\sigma \leq \tau$，$\mu(\sigma, \tau)$ 的值只与它们之间的结构差异有关，其数值形式就是带符号的第一类斯特林数的乘积形式。

3.3.6.2.3 3. 如何连接到斯特林反演？

我们通常看到的斯特林反演公式是关于 n 和 k 的，而不是关于具体分划 σ 的。这是因为斯特林反演是集合分划格反演的 “约化”（Reduced）形式。

第一步：从组合意义看 x^n (Zeta 变换)

考虑映射 $f : \{1, ..., n\} \to \{1, ..., x\}$。总共有 x^n 种映射。我们可以根据这个映射的 核（Kernel） 来分类。核是一个分划 σ，也就是把映射到同一个值的元素归为一个块。

如果我们设 $N(\sigma)$ 是“恰好以 σ 为核”的映射数量（即 σ 的每个块映射到不同的值）。那么总数 x^n 就是对所有可能的核 σ 求和（这就是 Zeta 变换的体现）：

$$x^n = \sum_{\sigma \in \Pi_n} N(\sigma)$$

注意到，如果分划 σ 有 k 个块（记为 $|\sigma| = k$），那么 $N(\sigma) = x^{\underline{k}}$（从 x 个值选 k 个排列）。而这也正好对应了有多少个分划具有 k 个块——这正是第二类斯特林数 $\left\{ {n \atop k} \right\}$。

所以上面的式子变成了我们熟悉的：

$$x^n = \sum_{k=0}^{n} \left\{ n \atop k \right\} x^{\underline{k}}$$

(这对应 ζ 变换，也就是 $g = \zeta * f$)

听不懂？见 A3.普通幂转下降幂的组合意义

第二步：反演 (Möbius 变换)

根据广义莫比乌斯反演原理，如果我们想求 $x^{\underline{n}}$（即强行要求核为 $\hat{0}$ 且全部映射到不同值的方案数），我们需要用 μ 函数倒推回去。

在“约化”的代数视角下，Π_n 的 Möbius 函数系数对应正是 第一类斯特林数。

具体来说，由于 $\mu(\hat{0}, \hat{1}) = (-1)^{n-1}(n-1)!$，这其实是第一类斯特林数 $s(n,1)$。 推广到从 n 个元素的块反演回 k 个元素的块，系数正是：

$$s(n,k) = (-1)^{n-k} \begin{bmatrix} n \\ k \end{bmatrix}$$

咋推广？ 对于 n 个元素的全部分划格 Π_n，从最底（全散）到最顶（全聚）的莫比乌斯函数是：

$$\mu_{\Pi_n}(\hat{0}, \hat{1}) = (-1)^{n-1}(n-1)!$$

那如果不是到最顶 $\hat{1}$，而是到一个中间状态 σ 呢？假设分划 σ 将 n 个元素分成了 k 个块，块的大小分别为 $b_1, b_2, ..., b_k$（显然 $\sum b_i = n$）。 关键性质： 区间 $[\hat{0}, \sigma]$ 在结构上等价于 k 个独立的小分划格的直积。

$$[\hat{0}, \sigma] \cong \Pi_{b_1} \times \Pi_{b_2} \times ... \times \Pi_{b_k}$$

因为莫比乌斯函数在直积上具有积性（Multiplicative），所以：

$$\mu(\hat{0}, \sigma) = \prod_{i=1}^{k} \mu_{\Pi_{b_i}}(\hat{0}, \hat{1})$$

代入基础结论：

$$\begin{aligned} \mu(\hat{0}, \sigma) &= \prod_{i=1}^{k} [(-1)^{b_i - 1}(b_i - 1)!] \\ &= (-1)^{\sum (b_i - 1)} \prod_{i=1}^{k} (b_i - 1)! \\ &= (-1)^{n-k} \prod_{i=1}^{k} (b_i - 1)! \end{aligned}$$

这是一个非常重要的中间结论：任意分划 σ 的莫比乌斯函数值，取决于它的块大小的阶乘积。我们在做斯特林反演时，不是针对某一个特定的 σ，而是把所有块数（Rank）为 k 的分划归为一类。所谓的“反演系数” $s(n,k)$，本质上就是把所有“这就只有 k 个块”的分划的 μ 值加起来：

$$s(n,k) = \sum_{\substack{\sigma \in \Pi_n \\ |\sigma| = k}} \mu(\hat{0}, \sigma)$$

代入刚才推导的公式：

$$s(n,k) = \sum_{\substack{\sigma \in \Pi_n \\ |\sigma| = k}} \left((-1)^{n-k} \prod_{i=1}^{k} (b_i - 1)! \right)$$

提公因式 $(-1)^{n-k}$：

$$s(n,k) = (-1)^{n-k} \underbrace{\sum_{\substack{\sigma \in \Pi_n \\ |\sigma| = k}} \left(\prod_{i=1}^{k} (b_i - 1)! \right)}_{\text{这是什么？}}$$

让我们看看这个求和部分的组合意义： 外部求和：枚举所有把 n 个元素分成 k 个块的方案（分划 σ）。 内部乘积 $\prod (b_i - 1)!$：对于每一个块（大小为 b_i），$(b_i - 1)!$ 正好是圆排列（Circle Permutation）的方案数。 合起来的意思是： 先选定一种分块方式，然后把每个块里的元素排成一个圆环（Cycle）。 把所有分块方式对应的圆环方案加起来，这不就是： “把 n 个元素排成 k 个圆环（轮换）的总方案数”吗？ 这正是无符号第一类斯特林数 $\begin{bmatrix} n \\ k \end{bmatrix}$ 的定义！ 所以：

$$s(n,k)=(-1)^{n-k}\begin{bmatrix}n\\k\end{bmatrix}$$

这就是为什么斯特林反演的系数里藏着圆排列的阶乘，也是它“推广”的内在逻辑。

所以反演公式为：

$$x^{\underline{n}}=\sum_{k=0}^{n}(-1)^{n-k}\begin{bmatrix}n\\k\end{bmatrix}x^k$$

(这对应 $\boldsymbol{f=\mu*g}$)

3.3.7 总结

反演名称	基础偏序集 (Poset)	偏序关系 $\leq$	$\zeta(x,y)$ (正向覆盖系数)	$\mu(x,y)$ (反演容斥系数)	卷积单位元 $\delta(x,y)$
通用定义	任意局部有限偏序集 P	$x\leq y$	1 若 $x\leq y$0 若不满足	ζ^{-1} (递归定义)	$[x=y]$
子集反演 (SOS DP)	布尔格 B_n (Power Set)	$S\subseteq T$	1	$(-1)^{\|T\|-\|S\|}$	$[S=T]$
超集反演 (高维后缀和)	布尔格 B_n (Dual)	$S\supseteq T$	1	$(-1)^{\|S\|-\|T\|}$	$[S=T]$
二项式反演 (PIE)	布尔格 (投影到大小)	大小 k 归类	$\binom{n}{k}$	$(-1)^{n-k}\binom{n}{k}$	$[n=k]$
数论莫比乌斯	整除格 D_n	$a\mid b$	1	$\mu(b/a)$	$[a=b]$
斯特林反演	分划格 Π_n	加细 (Refinement)	$\begin{Bmatrix}n\\k\end{Bmatrix}$	$(-1)^{n-k}\begin{bmatrix}n\\k\end{bmatrix}$	$[n=k]$

3.4 数论/数论笔记部分/数论笔记(和式变换)

3.4.1 和式变换规则与技术

3.4.1.1 基本变换规则

1. 分配律

$$\sum_{k\in K}ca_k=c\sum_{k\in K}a_k$$

2. 结合律

$$\sum_{k\in K}(a_k+b_k)=\sum_{k\in K}a_k+\sum_{k\in K}b_k$$

3. 交换律

$$\sum_{k\in K}a_k=\sum_{p(k)\in K}a_{p(k)}$$

其中 $p(k)$ 是指标集的任意排列

示例：$a_1+a_2+a_3+a_6=a_6+a_3+a_2+a_1$

3.4.1.2 高级变换技术

3.4.1.3 1. 替换条件式

$$\sum_{i=1}^{n}\sum_{j=1}^{m}\sum_{d\mid\gcd(i,j)}d=\sum_{i=1}^{n}\sum_{j=1}^{m}\sum_{d=1}^{\min(n,m)}[d\mid i][d\mid j]d$$

3.4.1.4 2. 替换指标变量

$$\sum_{i=1}^{n}\sum_{j=1}^{m}[\gcd(i,j)=k]=\sum_{i'=1}^{\lfloor n/k\rfloor}\sum_{j'=1}^{\lfloor m/k\rfloor}[\gcd(i',j')=1]$$

其中 $i'=i/k,j'=j/k$

3.4.1.5 3. 交换求和次序

$$\sum_{i=1}^{n}\sum_{j=1}^{m}A(i)B(j)=\sum_{j=1}^{m}\sum_{i=1}^{n}A(i)B(j)$$

一个比较好的方法是：列出变量的定义域，这样交换的时候可以通过定义域来推定别的变量，变量交换的目的是分离式子。

3.4.1.6 4. 分离变量

$$\sum_{i=1}^{n}\sum_{j=1}^{m}A(i)B(j)=\left(\sum_{i=1}^{n}A(i)\right)\left(\sum_{j=1}^{m}B(j)\right)$$

3.4.2 技巧

3.4.2.1 1. 区间整除条件式的封闭形式

$$\sum_{i=1}^{n}[k \mid i]=\lfloor\frac{n}{k}\rfloor$$

扩展

$$\lfloor\frac{\lfloor\frac{n}{k}\rfloor}{m}\rfloor=\lfloor\frac{n}{km}\rfloor$$

人话：在 1 到 n 的整数中，能被 k 整除的数的个数是 $\lfloor\frac{n}{k}\rfloor$

3.4.2.2 2. $[gcd(i,j)=1]$的进一步变换

$$\sum_{i=1}^{n}\sum_{j=1}^{m}[\gcd(i,j)=1]=\sum_{i=1}^{n}\sum_{j=1}^{m}\sum_{d\mid\gcd(i,j)}\mu(d)$$

$$=\sum_{i=1}^{n}\sum_{j=1}^{m}\sum_{d=1}^{\min(n,m)}\mu(d)[d\mid i][d\mid j]$$

人话：变换枚举顺序，d 能整除 i 和 j，则 d 能整除 gcd(i,j)

3.4.2.3 3. 有序对和无序对求和的互相转换

$$\sum_{i=1}^{n}\sum_{j=1}^{n}A(i,j)=2*\sum_{i=1}^{n}\sum_{j=i}^{n}A(i,j)-\sum_{i=1}^{n}A(i,i)$$

当且仅当 $A(i,j)=A(j,i)$ 时成立

人话：将有序对转换为无序对，注意枚举顺序

3.4.2.4 4. 通过$gcd(i,j)=1$构造条件式$[gcd(i,j)=1]$

令

$$d=\gcd(i,j),i=i'd,j=j'd$$

注意，当且仅当

$$\gcd(i',j')=1$$

时，此变换成立，于是

$$\sum_{i=1}^{n}\sum_{j=1}^{m}f(\gcd(i,j))$$

$$=\sum_{d=1}^{\min(n,m)}\sum_{i'd=1}^{n}\sum_{j'd=1}^{m}f(d)[\gcd(i',j')=1]$$

$$=\sum_{d=1}^{\min(n,m)}\sum_{i'=1}^{\lfloor n/d\rfloor}\sum_{j'=1}^{\lfloor m/d\rfloor}f(d)[\gcd(i',j')=1]$$

变量换名

$$=\sum_{d=1}^{\min(n,m)}\sum_{i=1}^{\lfloor n/d\rfloor}\sum_{j=1}^{\lfloor m/d\rfloor}f(d)[\gcd(i,j)=1]$$

人话：没有人话

3.4.2.5 5. $gcd(x,y)$的变换

$$n=\sum_{d\mid n}\varphi(d)$$

$$gcd(x,y)=\sum_{d\mid\gcd(x,y)}\varphi(d)=\sum_{d\mid x,d\mid y}\varphi(d)$$

4 组合与生成函数

4.1 数论/数论笔记部分/数论笔记(排列组合)

4.1.1 圆排列

n 个不同元素围成一圈的圆排列数，记作 Q_n^n。

考虑其中已经排好的一圈，从不同位置断开，会变成 n 个不同的线排列：

$$Q_n^n \times n = A_n^n$$

则

$$Q_n^n = \frac{A_n^n}{n} = (n-1)!$$

例如，3 个不同元素的圆排列数为 $(3-1)! = 2$ 种：

从 n 个不同元素中选 m 个围成一圈的圆排列数，记作 Q_n^m：

$$Q_n^m = C_n^m \cdot Q_m^m = \frac{n!}{m \cdot (n-m)!}$$

> 其实就是全排列固定了一个数

4.1.2 错位排列

错位排列是没有任何元素出现在其有序位置的排列。对于 $1 \sim n$ 的排列 P，如果满足 $P_i \neq i$，则称 P 是 n 的错位排列。

4.1.2.1 错位排列数

- $D_1 = 0$
- $D_2 = 1$（即 $\{2,1\}$）
- $D_3 = 2$（即 $\{2,3,1\}, \{3,1,2\}$）
- $D_4 = 9$（即 $\{2,1,4,3\}, \{2,3,4,1\}, \{2,4,1,3\}, \{3,1,4,2\}, \{3,4,1,2\}, \{3,4,2,1\}, \{4,1,2,3\}, \{4,3,1,2\}, \{4,3,2,1\}$）

4.1.2.2 递推关系 $D_n = (n-1)(D_{n-1} + D_{n-2})$

边界条件：

$$D_1 = 0, \quad D_2 = 1$$

n	1	2	3	4	5	6	7	8
D_n	0	1	2	9	44	265	1854	14833

4.1.2.3 信封问题

n 封不同的信（编号 $1, 2, ..., n$）放入 n 个编号对应的信封中，要求每个信封的编号与信的编号都不相同。有多少种放置方法？

4.1.2.4 递推关系分析

考虑第 n 封信的放置：
1. 情况 1：前 $n-1$ 封信已全错排 - 第 n 封信只需与前面任一封信交换位置 - 方法数：$(n-1) \cdot D_{n-1}$

2. 情况 2：前 $n-1$ 封信恰好有 1 封位置正确
   - 第 n 封信必须与位置正确的信交换
   - 方法数：$(n-1) \cdot D_{n-2}$

> 其他情况无法通过一次交换变成全错排

4.1.3 第一类斯特林数（斯特林轮换数）

将 n 个不同元素划分为 m 个非空圆排列的方案数， 记作 $S(n,m)$ 或：

$$\begin{bmatrix} n \\ m \end{bmatrix}$$

4.1.3.1 递推关系

$$\begin{bmatrix} n \\ m \end{bmatrix} = \begin{bmatrix} n-1 \\ m-1 \end{bmatrix} + (n-1) \begin{bmatrix} n-1 \\ m \end{bmatrix}$$

4.1.3.2 组合解释（圆桌问题）

n 个人坐 m 张圆桌的方案数，考虑第 n 个人的两种坐法：

1. 单独坐一桌
   - 前 $n-1$ 人坐满剩余的 $m-1$ 张桌
   - 方案数：$\begin{bmatrix} n-1 \\ m-1 \end{bmatrix}$
2. 与其他人同坐
   - 前 $n-1$ 人先坐满 m 张桌
   - 第 n 人可坐到任意 $n-1$ 个人的左侧
   - 方案数：$(n-1) \cdot \begin{bmatrix} n-1 \\ m \end{bmatrix}$

4.1.4 第二类斯特林数（斯特林子集数）
将 n 个不同元素划分为 m 个非空子集的方案数， 记作 $S(n,m)$ 或：

$$\begin{Bmatrix} n \\ m \end{Bmatrix}$$

4.1.4.1 递推关系

$$\begin{Bmatrix} n \\ m \end{Bmatrix} = \begin{Bmatrix} n-1 \\ m-1 \end{Bmatrix} + m\begin{Bmatrix} n-1 \\ m \end{Bmatrix}$$

4.1.4.2 组合解释（房间分配问题）
n 个人进入 m 个房间的方案数（每个房间非空），考虑第 n 个人的两种选择：

1. 单独进入新房间
   - 前 $n-1$ 人进入剩余的 $m-1$ 个房间
   - 方案数：$\begin{Bmatrix} n-1 \\ m-1 \end{Bmatrix}$
2. 进入已有人的房间
   - 前 $n-1$ 人先进入所有 m 个房间
   - 第 n 人可选择进入任意一个已有人的房间
   - 方案数：$m \cdot \begin{Bmatrix} n-1 \\ m \end{Bmatrix}$

4.1.5 Catalan 数 通项公式
(1) $H_n = C_{2n}^n - C_{2n}^{n-1}$
(2) $H_n = \frac{1}{n+1}C_{2n}^n$
(3) $H_n = \frac{4n-2}{n+1}H_{n-1}$

4.1.5.1 证明

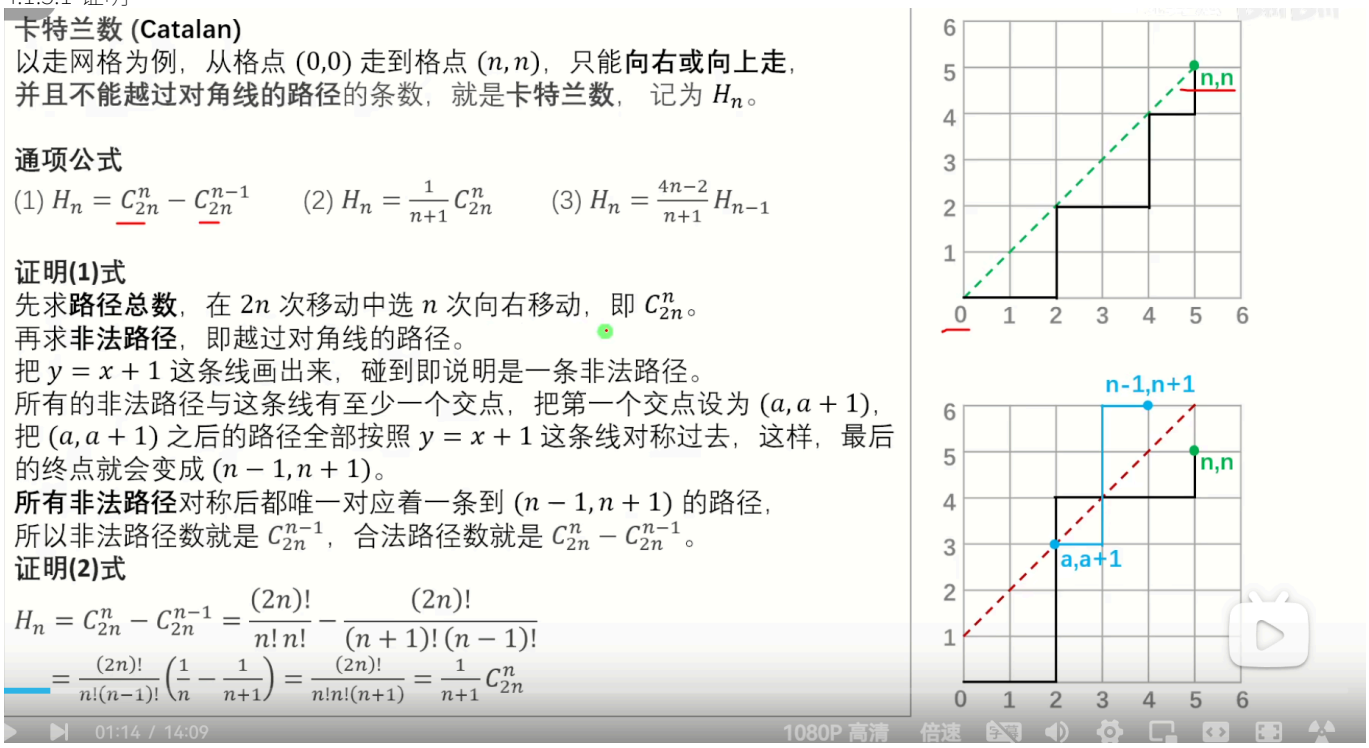


图 1　本地图片

4.1.5.2 Catalan 数列

H_0	H_1	H_2	H_3	H_4	H_5	H_6	H_7	H_8
1	1	2	5	14	42	132	429	1430

4.1.5.3 Catalan 特征
从 (0,0) 到 (n,n)，不越过对角线，即任何时候，向上走的步数不能超过向右走的步数。 一种操作数不能超过另外一种操作数，或者两种操作不能有交集，这些操作的合法方案数，通常是卡特兰数。

4.1.5.4 Catalan 应用
1. 一个有 n 个 0 和 n 个 1 组成的字串，且所有的前缀字串皆满足 1 的个数不超过 0 的个数。这样的字串个数有多少?
2. 包含 n 组括号的合法运算式的个数有多少?
3. 一个栈的进栈序列为 1,2,3, $\cdots$, n，有多少个不同的出栈序列? 合法性：任何时刻不能空栈出栈 ⇒ 任意前缀 “)” 不多于 “(”；最后入栈、出栈各 n 次，栈空。
4. n 个结点可构造多少个不同的二叉树?
5. 在圆上选择 $2n$ 个点，将这些点成对连接起来使得所得的 n 条弦不相交的方法数?
6. 通过连结顶点而将 $n+2$ 边的凸多边形分成 n 个三角形的方法数?

4.1.5.5 说明
这些都是卡特兰数，因为它们都与合法括号序列 / Dyck 路径存在天然的双射，或者都满足同一个卡特兰递推

$$C_0 = 1, \qquad C_n = \sum_{i=0}^{n-1} C_i\, C_{n-1-i}\ (n \geq 1),$$

并且初值一致，所以计数相同。分别说——

4.1.5.6 3) 单栈出栈序列的个数 = C_n
把栈的操作写成长度 $2n$ 的串：- 入栈记“(”，出栈记“)”。 合法性：任何时刻不能空栈出栈 ⇒ 任意前缀“)”不多于“(”；最后入栈、出栈各 n 次。 这正是合法括号序列的定义，因此个数为 $C_n = \frac{1}{n+1}\binom{2n}{n}$。

（同说法：栈可生成的排列=231-避免排列，其数目为卡特兰数。）

4.1.5.7 4) n 个结点的二叉树个数 = C_n
设 T_n 为有 n 个结点（有序、无标号）的二叉树数。以根为界：左子树 i 个结点、右子树 $n-1-i$ 个结点，二者独立：

$$T_n = \sum_{i=0}^{n-1} T_i\, T_{n-1-i}, \quad T_0 = 1.$$

这恰是卡特兰递推，因此 $T_n = C_n$。（等价双射：对二叉树做先序/中序边走访，“向下”记“(”，“返回”记“)”，得到 Dyck 串，反之亦然。）

4.1.5.8 5) 圆上 $2n$ 点配对且弦不相交的配法数 = C_n
固定点 1，它必须与某个点 $2k$ 相连（顺时针计）。这一条弦把圆分成两侧：- 一侧有 $2k-2$ 个点，可不交配对数 C_{k-1}；- 另一侧有 $2n-2k$ 个点，可不交配对数 C_{n-k}。 枚举 $k = 1..n$ 得

$$M_n = \sum_{k=1}^{n} C_{k-1} C_{n-k} = \sum_{i=0}^{n-1} C_i C_{n-1-i}.$$

同初值 $M_0 = 1$，故 $M_n = C_n$。

4.1.5.9 6) 将凸 $(n+2)$-边形三角剖分的方法数 = C_n
固定顶点 1，选一条对角线 $(1, j)$（$j = 3..n+2$）。它把多边形分成：- 一个 $(j-1)$-边形（可三角剖分数 C_{j-3}）；- 一个 $(n+3-j)$-边形（可三角剖分数 $C_{n-(j-2)}$）。 求和得同一递推：

$$T_n = \sum_{j=3}^{n+2} C_{j-3} C_{n-(j-2)} = \sum_{i=0}^{n-1} C_i C_{n-1-i}, \quad T_0 = 1,$$

故 $T_n = C_n$。

4.1.5.10 递推来源
4.1.5.11 从合法括号序列（Dyck 路径）推导
考虑长度 $2n$ 的合法括号序列：

- 第一个符号必然是“(”。
- 找到与之匹配的“)”位置。假设它在第 $2k+2$ 个位置（即括号包住了 k 对括号）。

于是序列可以分成三部分：

$$(\ \underbrace{S}_{2k\ \text{长度的合法串}}\ )\ \underbrace{T}_{2(n-k-1)\ \text{长度的合法串}}$$

- 内部 S 是一个合法串，长度 $2k$，个数 C_k；
- 外部 T 也是一个合法串，长度 $2(n-k-1)$，个数 C_{n-1-k}。

所以

$$C_n = \sum_{k=0}^{n-1} C_k \cdot C_{n-1-k}.$$

这就是递推的来源。

4.2 数论/数论笔记部分/数论笔记(排列组合进阶)

4.2.1 组合意义天地灭 代数推导保平安

主要记录一些好玩的小玩意 特别感谢 cbdsopa 的博客-OI 中基本的组合数学公式/oi_wiki 组合数学

比较深刻的是 如果两个计数过程本质相同，那么答案相同，即使过程不同

4.2.2 插板法

插板法是用于求一类给相同元素分组的方案数的一种技巧，也可以用于求一类线性不定方程的解的组数。
4.2.2.1 正整数和的数目
问题一：现有 n 个 完全相同 的元素，要求将其分为 k 组，保证每组至少有一个元素，一共有多少种分法？

考虑拿 $k-1$ 块板子插入到 n 个元素两两形成的 $n-1$ 个空里面。

因为元素是完全相同的，所以答案就是 $\binom{n-1}{k-1}$。

本质是求 $x_1+x_2+\cdots+x_k=n$ 的正整数解的组数。

4.2.2.2 非负整数和的数目

问题二：现有 n 个 完全相同 的元素，要求将其分为 k 组，每组可以为空，一共有多少种分法?

我们考虑创造条件转化成有限制的问题一，先借 k 个元素过来，在这 $n+k$ 个元素形成的 $n+k-1$ 个空里面插板，答案为

$$\binom{n+k-1}{k-1}=\binom{n+k-1}{n}$$

组合意义：

开头我们借来了 k 个元素，用于保证每组至少有一个元素，插完板之后再把这 k 个借来的元素从 k 组里面拿走。因为元素是相同的，所以转化过的情况和转化前的情况可以一一对应，答案也就是相等的。

由此可以推导出插板法的公式：$\binom{n+k-1}{n}$。

本质是求 $x_1+x_2+\cdots+x_k=n$ 的非负整数解的组数（即要求 $x_i\geq 0$）。

4.2.2.3 不同下界整数和的数目

问题三：现有 n 个 完全相同 的元素，要求将其分为 k 组，对于第 i 组，至少要分到 $a_i,\sum a_i\leq n$ 个元素

本质是求 $x_1+x_2+\cdots+x_k=n$ 的解的数目，其中 $x_i\geq a_i$。

类比无限制的情况，我们借 $\sum a_i$ 个元素过来，保证第 i 组至少能分到 a_i 个。也就是令

$$x_{i'}=x_i-a_i$$

得到新方程：

$$\begin{aligned}(x_{1'}+a_1)+(x_{2'}+a_2)+\cdots+(x_{k'}+a_k)&=n\\ x_{1'}+x_{2'}+\cdots+x_{k'}&=n-a_1-a_2-\cdots-a_k\\ x_{1'}+x_{2'}+\cdots+x_{k'}&=n-\sum a_i\end{aligned}$$

其中

$$x_{i'}\geq 0$$

然后问题三就转化成了问题二，直接用插板法公式得到答案为

$$\binom{n-\sum a_i+k-1}{n-\sum a_i}$$

4.2.2.4 不相邻的排列

$1\sim n$ 这 n 个自然数中选 k 个，这 k 个数中任何两个数都不相邻的组合有 $\binom{n-k+1}{k}$ 种。

组合意义：

假设我们已经完成了选择。那么这 n 个数就被分成了两类： - 被选中的数 - 没被选中的数

我们考虑把$n-k$没被选中的数分别排成一列,产生了 $n-k+1$ 个空,然后在这 $n-k+1$ 个空里面插入 k 个板子 ,答案就是 $\binom{n-k+1}{k}$ 种。

构造双射：

问题等价于 我们需要从 $1,2,...,n$ 中选出 k 个数，设这就 k 个数从小到大排序后为 $a_1,a_2,...,a_k$。 满足：

$$1\leq a_1<a_2<...<a_k\leq n$$

且

$$a_{i+1}-a_i\geq 2\quad(1\leq i<k)$$

考虑构造

$$b_i=a_i-(i-1)$$

原式变为

$$b_i<b_{i+1}$$

b_i 上界

$$b_k=a_k-(k-1)\leq n-(k-1)=n-k+1$$

经过变换后，原问题等价于： 从整数集合 $\{1,2,...,n-k+1\}$ 中选出 k 个互不相同的整数 $b_1<b_2<...<b_k$ 这个问题是简单的，请读者自己算

4.2.3 组合恒等式
4.2.3.1 1. 对称式

$$\binom{n}{m}=\binom{n}{n-m}$$

组合意义: 你 n 中拿 m 个等价于有 $n-m$ 个不拿。
4.2.3.2 2. 二项式定理/多项式定理

$$(x+y)^n=\sum_{i=0}^{n}\binom{n}{i}x^iy^{n-i}$$

组合意义： 我们把 $(x+y)^n$ 看作是有 n 个 $(x+y)$ 相乘，那么得到一个 x^ay^{n-a} 相当于是从 n 个 $(x+y)$ 中选出 a 个 $(x+y)$ 中的 x 相乘，那么结果的多项式中就有一项是 $\binom{n}{a}x^ay^{n-a}$ 。所有的这种项都满足这种情况，那么公式可得。

其实不一定要求必须是两元的，多元的也是同理。 然后我们可以得到多项式定理：

$$(x_1+x_2+...+x_m)^n=\sum_{n_1+n_2+...+n_m=n}\binom{n}{n_1,n_2,...,n_m}x_1^{n_1}x_2^{n_2}\cdots x_m^{n_m}$$

证明： 同二项式定理证明，对于 $x_1^{n_1}x_2^{n_2}\cdots x_m^{n_m}$ 这一项的系数为 $\binom{n}{n_1}\binom{n-n_1}{n_2}\cdots\binom{n-n_1-n_2-\ldots-n_{m-1}}{n_m}$ 。 展开发现可以抵消一些东西，于是上面的系数就等于：

$$\frac{n!}{n_1!n_2!\cdots n_m!}=\binom{n}{n_1,n_2,...,n_m}$$

其实这个式子就是多重集排列数
4.2.3.3 3. 帕斯卡定理

$$\binom{n}{k}=\binom{n-1}{k-1}+\binom{n-1}{k}$$

我们知道 $(x+y)^n$ 得到的多项式是系数满足杨辉三角的，我们知道了二项式定理的话，这个东西就是杨辉三角的递推式。

组合意义： 考虑已经选了$n-1$个数，现在要选第n个数，那么第n个数选了的话，就相当于从$n-1$个数中选$k-1$个数，第n个数没选的话，就相当于从$n-1$个数中选k个数。
4.2.3.4 4. 特殊的二项式定理/第一类二项式反演

$$\binom{n}{0}+\binom{n}{1}+...+\binom{n}{n}=2^n$$

实际上是 $(1+1)^n$ ， 当然也可以从组合意义上证明 ， 当然这是不必要的。

$$\sum_{i=0}^{n}(-1)^i\binom{n}{i}=[n=0]$$

实际上是 $(1-1)^n$ ， 我称之为第一类二项式反演。
4.2.3.5 5. 递推式 1

$$\binom{n}{k}=\frac{n}{k}\binom{n-1}{k-1}$$

按定义来就没了，简单提一下。

代数推导：

$$\begin{aligned}\binom{n}{k}&=\frac{n!}{k!(n-k)!}\\&=\frac{n!}{(k-1)!(n-k)!}\cdot\frac{n}{k}\\&=\frac{n}{k}\binom{n-1}{k-1}\end{aligned}$$

这个式子比较深刻 比如，要求解这个经典求和：

$$S=\sum_{k=0}^{n}k\binom{n}{k}$$

直接算很难，因为系数里有个 k。利用 $k\binom{n}{k}=n\binom{n-1}{k-1}$，我们可以把 k “吸”进组合数里，把常数 n “吐”出来：

$$\begin{aligned}\sum_{k=1}^{n}k\binom{n}{k}&=\sum_{k=1}^{n}n\binom{n-1}{k-1}\\&=n\sum_{k=1}^{n}\binom{n-1}{k-1}\quad(\text{令}\ \ j=k-1)\\&=n\sum_{j=0}^{n-1}\binom{n-1}{j}\\&=n\cdot2^{n-1}\end{aligned}$$

这个式子正是 7.变下项求和式中的第一个式子。

4.2.3.6 6. 积式

$$\binom{n}{r}\binom{r}{k}=\binom{n}{k}\binom{n-k}{r-k}$$

定义展开左边上下分子分母同乘 $(n-k)!$ 即可证明。

代数推导：

$$\begin{aligned}\binom{n}{r}\binom{r}{k}&=\frac{n!}{r!(n-r)!}\cdot\frac{r!}{k!(r-k)!}\\&=\frac{n!}{k!(n-k)!}\cdot\frac{(n-k)!}{(r-k)!(n-r)!}\\&=\binom{n}{k}\binom{n-k}{r-k}\end{aligned}$$

4.2.3.7 7. 变下项求和式

$$\sum_{k=0}^{n}k\binom{n}{k}=n2^{n-1}$$

代数推导：

$$\begin{aligned}\sum_{k=0}^{n}k\binom{n}{k}&=\sum_{k=1}^{n}k\binom{n}{k}\\&=\sum_{k=1}^{n}n\binom{n-1}{k-1}\\&=n\sum_{i=0}^{n-1}\binom{n-1}{i}\\&=n2^{n-1}\end{aligned}$$

从二项式定理的求导也可以推出 利用二项式定理：

$$(1+x)^n=\sum_{k=0}^{n}\binom{n}{k}x^k$$

两边对 x 求导：

$$n(1+x)^{n-1}=\sum_{k=0}^{n}k\binom{n}{k}x^{k-1}$$

令 $x=1$，代入得：

$$n(1+1)^{n-1}=\sum_{k=0}^{n}k\binom{n}{k}(1)^{k-1}$$

$$n2^{n-1}=\sum_{k=0}^{n}k\binom{n}{k}$$

$$\sum_{k=0}^{n}k^2\binom{n}{k}=n(n+1)2^{n-2}$$

证明和上面差不多，就不证了。

4.2.3.8 8. 变上项求和式

$$\sum_{l=0}^{n}\binom{l}{k}=\binom{n+1}{k+1}$$

组合意义：指定 $n+1$ 个数的集合 $S=\{a_1,a_2,...,a_{n+1}\}$。先考虑右边的组合意义，即从 $n+1$ 个数中选出 $k+1$ 个。左边的组合意义：相当于是总共 $n+1$ 种不累加的情况的加法原理。

第一种：指定必须选择 a_1，然后从剩余的 n 个数中选择 k 个，方案数 $\binom{n}{k}$。

第二种：指定必须不选择 a_1 而选择 a_2，然后从剩余的 $n-1$ 个数中选择 k 个，方案数 $\binom{n-1}{k}$。

第三种：指定必须不选择 a_1,a_2 而选择 a_3，然后从剩余的 $n-2$ 个数中选择 k 个，方案数 $\binom{n-2}{k}$。

$$\vdots$$

第 $n+1$ 种：指定必须不选择 $a_1,a_2,...,a_n$ 而选择 a_{n+1}，然后从剩余的 0 个数中选择 k 个，方案数 $\binom{0}{k}$。

总体的组合意义 $\sum_{i=0}^{n}\binom{i}{k}$ 等价于从 $n+1$ 个数中选出 $k+1$ 个，那么等式左右两边组合意义相同，等式成立。

4.2.3.9 8.1 变上项求和式 (扩展)

$$\sum_{i=0}^{n}\binom{i+m}{m}=\binom{n+m+1}{m+1}$$

另外的一种常用的形式是上下项共变的。 证明和上面的式子是类似的。 但是我们采取代数推导：

$$S=\underbrace{\binom{m+1}{m+1}}_{原\ \binom{m}{m}}+\binom{m+1}{m}+\binom{m+2}{m}+...+\binom{n+m}{m}$$

$$S=\underbrace{\binom{m+2}{m+1}+\binom{m+2}{m}}_{合并这两项}+...+\binom{n+m}{m}$$

一直合并 Q.E.D.

4.2.3.10 9. 积和式 (范德蒙德卷积)

$$\sum_{k=0}^{r}\binom{m}{k}\binom{n}{r-k}=\binom{m+n}{r},\quad r\le\min\{n,m\}$$

组合意义：指定集合 $A=\{a_1,a_2,...,a_m\}$ 和 $B=\{b_1,b_2,...,b_n\}$。右边问题等价于从 A,B 两个集合中选出 r 个的方案数。左边问题等价于：对于 $k\in[0,r]$，先在 A 中先取出 k 个，然后在 B 中选出 $r-k$ 个的总方案数。即 A,B 中总共选出 r 个的方案数。所以左右两个问题组合意义等价，等式成立。

$$\sum_{i=0}^{n}\binom{n}{i}^2=\binom{2n}{n}$$

这是 9.积和式 的一种特殊情况，代入 $m=n$ 即可。

$$\sum_{k=0}^{r}\binom{k}{a}\binom{r-k}{b}=\binom{r+1}{a+b+1}$$

几何意义：指定集合 $S=\{a_1,a_2,...,a_{r+1}\}$。右边问题等价于从 S 中选出 $a+b+1$ 个数。左边问题等价于：第 i 种，指定选出 a_i，然后再在 $[1,i-1]$ 和 $[i+1,r]$ 中分别选出 a 个和 b 个。于是这几种合并起来就等价于 $\binom{r+1}{a+b+1}$。因此左右两边组合意义等价，等式成立。

为啥不重复？我们考虑 S 是升序的 a_i，即为第 $a+1$ 小，我们钦定 i 的时候保证了独立性

4.2.3.11 10. 第二类二项式反演

$$f(n)=\sum_{i=0}^{n}(-1)^i\binom{n}{i}g(i)\Leftrightarrow g(n)=\sum_{i=0}^{n}(-1)^i\binom{n}{i}f(i)$$

代数推导：已知：$g(n)=\sum_{i=0}^{n}(-1)^i\binom{n}{i}f(i)$ 则：

$$\begin{aligned}
\sum_{i=0}^{n}(-1)^i\binom{n}{i}g(i)&=\sum_{i=0}^{n}(-1)^i\binom{n}{i}\sum_{j=0}^{i}(-1)^j\binom{i}{j}f(j)\\
&=\sum_{i=0}^{n}\sum_{j=0}^{i}(-1)^{i+j}\binom{n}{i}\binom{i}{j}f(j)\\
&=\sum_{i=0}^{n}\sum_{j=0}^{i}(-1)^{i+j}\binom{n}{j}\binom{n-j}{i-j}f(j)\\
&=\sum_{j=0}^{n}(-1)^j\binom{n}{j}f(j)\sum_{i=j}^{n}(-1)^i\binom{n-j}{i-j}\\
&=\sum_{j=0}^{n}(-1)^j\binom{n}{j}f(j)\sum_{i=0}^{n-j}(-1)^{i+j}\binom{n-j}{i}\\
&=\sum_{j=0}^{n}\binom{n}{j}f(j)\sum_{i=0}^{n-j}(-1)^i(-1)^{2j}\binom{n-j}{i}\\
&=\sum_{j=0}^{n}\binom{n}{j}f(j)\sum_{i=0}^{n-j}(-1)^i\binom{n-j}{i}\\
&=\sum_{j=0}^{n}\binom{n}{j}f(j)[n-j=0]\\
&=f(n)
\end{aligned}$$

积式->变换求和顺序->第一类二项式反演

然后我们还可以得到一个扩展：

$$f(n)=\sum_{i=0}^{n}\binom{n}{i}g(i)\Leftrightarrow g(n)=\sum_{i=0}^{n}(-1)^{n-i}\binom{n}{i}f(i)$$

证明： 令 $G(n)=(-1)^n g(n)$ ，有： 若 $f(n)=\sum_{i=0}^{n}(-1)^i\binom{n}{i}G(i)$ 成立，有 $G(n)=\sum_{i=0}^{n}(-1)^i\binom{n}{i}f(i)$。 即

$$G(n)=\sum_{i=0}^{n}(-1)^i\binom{n}{i}f(i)$$

$$\therefore g(n)(-1)^n=\sum_{i=0}^{n}(-1)^i\binom{n}{i}f(i)$$

当 n 为偶数，则 $n-i$ 与 i 奇偶性相同，有 $g(n)=\sum_{i=0}^{n}(-1)^{n-i}\binom{n}{i}f(i)$ 当 n 为奇数，则 $n-i$ 与 i 奇偶性相反，同样有 $g(n)=\sum_{i=0}^{n}(-1)^{n-i}\binom{n}{i}f(i)$ 由此有 $g(n)=\sum_{i=0}^{n}(-1)^{n-i}\binom{n}{i}f(i)$

4.2.3.12 11.浅对角线求和

$$\sum_{i=0}^{n}\binom{n-i}{i}=F_{n+1}$$

组合意义：爬楼梯 (Climbing Stairs)问题： 假设你要爬一个 n 级台阶的楼梯。你每次只能走 1 阶 或 2 阶。问有多少种不同的爬法? 角度 A：动态规划（对应右边 F_{n+1}）设 $f(n)$ 为爬 n 级台阶的方法数。 最后一步可能是迈了 1 阶（前一步在 $n-1$），或者是迈了 2 阶（前一步在 $n-2$）。 递推公式：$f(n)=f(n-1)+f(n-2)$。初始值：$f(0)=1$ (不动也是一种), $f(1)=1$。这是标准的斐波那契数列定义。所以总方案数对应 F_{n+1}。 角度 B：枚举“迈 2 阶”的次数（对应左边 $\sum$）我们换一种数法：按“一共迈了几次 2 阶”来分类。假设我们在整个过程中，一共迈了 i 次 2 阶。消耗台阶数：这 i 次 2 阶共消耗了 $2i$ 级台阶。剩余台阶数：剩下的 $n-2i$ 级台阶，必须全部由 1 阶 走完（共 $n-2i$ 次 1 阶）。 总步数：

$$\text{总步数}=(2\text{ 阶的次数})+(1\text{ 阶的次数})=i+(n-2i)=n-i$$

排列组合：我们要在这 $n-i$ 步中，选出哪 i 步是走“2 阶”的。这相当于从 $n-i$ 个位置中选 i 个位置。方案数为：$\binom{n-i}{i}$。结论既然所有的方案就是枚举 i 从 0 到最大可能值（$\lfloor n/2\rfloor$），把这些情况加起来，就是总的爬楼梯方案数。

$$\sum_{i}\binom{n-i}{i}=\text{爬 n 阶楼梯的总方案数}=F_{n+1}$$

4.2.4 多重集的组合数 1

设 $S=\{n_1\cdot a_1,n_2\cdot a_2,\cdots,n_k\cdot a_k\}$ 表示由 n_1 个 a_1，n_2 个 a_2，$\cdots$，n_k 个 a_k 组成的多重集。那么对于整数 $r(r<n_i,\forall i\in[1,k])$，从 S 中选择 r 个元素组成一个多重集的方案数就是 多重集的组合数。这个问题等价于 $x_1+x_2+\cdots+x_k=r$ 的非负整数解的数目，可以用插板法解决，答案为

$$\binom{r+k-1}{k-1}$$

4.2.5 多重集的组合数 2

考虑这个问题：设 $S=\{n_1\cdot a_1,n_2\cdot a_2,\cdots,n_k\cdot a_k,\}$ 表示由 n_1 个 a_1，n_2 个 a_2，$\cdots$，n_k 个 a_k 组成的多重集。那么对于正整数 r，从 S 中选择 r 个元素组成一个多重集的方案数。

这样就限制了每种元素的取的个数。同样的，我们可以把这个问题转化为带限制的线性方程求解：

$$\forall i\in[1,k],x_i\le n_i,\sum_{i=1}^{k}x_i=r$$

于是很自然地想到了容斥原理。容斥的模型如下：

1. 全集：$\sum_{i=1}^{k}x_i=r$ 的非负整数解。
2. 属性：$x_i\le n_i$。

于是设满足属性 i 的集合是 S_i，$\overline{S_i}$ 表示不满足属性 i 的集合，即满足 $x_i\ge n_i+1$ 的集合（转化为上面插板法的问题三）。

怎么转化的？ e.g. 我们强制分配 n_i+1 个 a_i ，剩下的 $r-(n_i+1)$ 个 a_i 可以任意取，所以方案数为 $\binom{r-(n_i+1)+k-1}{k-1}$

那么答案即为

$$\left|\bigcap_{i=1}^{k}S_i\right|=|U|-\left|\bigcup_{i=1}^{k}\overline{S_i}\right|$$

根据容斥原理，有：

$$\begin{aligned}\left|\bigcup_{i=1}^{k}\overline{S_i}\right| &= \sum_i\left|\overline{S_i}\right| - \sum_{i,j}\left|\overline{S_i}\cap\overline{S_j}\right| + \sum_{i,j,k}\left|\overline{S_i}\cap\overline{S_j}\cap\overline{S_k}\right| - \cdots \\ &\quad +(-1)^{k-1}\left|\bigcap_{i=1}^{k}\overline{S_i}\right| \\ &= \sum_i\binom{k+r-n_i-2}{k-1} - \sum_{i,j}\binom{k+r-n_i-n_j-3}{k-1} + \sum_{i,j,k}\binom{k+r-n_i-n_j-n_k-4}{k-1} - \cdots \\ &\quad +(-1)^{k-1}\binom{k+r-\sum_{i=1}^{k}n_i-k-1}{k-1}\end{aligned}$$

拿全集 $|U| = \binom{k+r-1}{k-1}$ 减去上式，得到多重集的组合数

$$Ans = \sum_{p=0}^{k}(-1)^p\sum_{A}\binom{k+r-1-\sum_A n_{A_i}-p}{k-1}$$

其中 A 是充当枚举子集的作用，满足 $|A| = p, A_i < A_{i+1}$。

4.3 数论/数论笔记部分/数论笔记(生成函数)

4.3.1 普通生成函数

序列 a 的普通生成函数（ordinary generating function，OGF）定义为形式幂级数（其实就是一个多项式()）：

$$F(x) = \sum_n a_n x^n$$

a 既可以是有穷序列，也可以是无穷序列。常见的例子（假设 a 以 0 为起点）：

1. 序列 $a = \langle 1, 2, 3\rangle$ 的普通生成函数是 $1 + 2x + 3x^2$。
2. 序列 $a = \langle 1, 1, 1, \cdots\rangle$ 的普通生成函数是 $\sum_{n\geq 0} x^n$。
3. 序列 $a = \langle 1, 2, 4, 8, 16, \cdots\rangle$ 的生成函数是 $\sum_{n\geq 0} 2^n x^n$。
4. 序列 $a = \langle 1, 3, 5, 7, 9, \cdots\rangle$ 的生成函数是 $\sum_{n\geq 0}(2n+1)x^n$。

换句话说，如果序列 a 有通项公式，那么它的普通生成函数的系数就是通项公式。

4.3.2 基本运算

考虑两个序列 a, b 的普通生成函数，分别为 $F(x), G(x)$。那么有

$$F(x) \pm G(x) = \sum_n (a_n \pm b_n)x^n$$

因此 $F(x) \pm G(x)$ 是序列 $\langle a_n \pm b_n\rangle$ 的普通生成函数。

考虑乘法运算，也就是卷积：

$$F(x)G(x) = \sum_n x^n \sum_{i=0}^{n} a_i b_{n-i}$$

因此 $F(x)G(x)$ 是序列 $\langle \sum_{i=0}^{n} a_i b_{n-i}\rangle$ 的普通生成函数。

4.3.3 封闭形式

在运用生成函数的过程中，我们不会一直使用形式幂级数的形式，而会适时地转化为封闭形式以更好地化简。

例如 $\langle 1, 1, 1, \cdots\rangle$ 的普通生成函数 $F(x) = \sum_{n\geq 0} x^n$，我们可以发现

$$F(x)x + 1 = F(x)$$

那么解这个方程得到

$$F(x) = \frac{1}{1-x}$$

这就是 $\sum_{n\geq 0} x^n$ 的封闭形式。

考虑等比数列 $\langle 1, p, p^2, p^3, p^4, \cdots\rangle$ 的生成函数 $F(x) = \sum_{n\geq 0} p^n x^n$，有

$$\begin{aligned}F(x)px + 1 &= F(x) \\ F(x) &= \frac{1}{1-px}\end{aligned}$$

等比数列的封闭形式与展开形式是常用的变换手段。

4.3.4 应用

接下来给出一些例题，来介绍生成函数在 OI 中的具体应用。

4.3.4.1 普通生成函数可以用来解决多重集合组合数问题。

问题：有 n 种物品，每种物品有 a_i 个，问取 m 个物品的组合数？

4.3.4.2 多重集合组合数
设从每种物品中取 b_i 个，$0 \le b_i \le a_i$， $m = \sum_{i=1}^{n} b_i$，对于一组选定的 b_i 进行组合的方案数为 1。 例如，取 3 个 A，1 个 B 的方案就是 {AAAB}；取 2 个 A、2 个 B 的方案就是 {AABB}。 那么，所有满足 $b_1 + b_2 + \cdots + b_n = m$ 的方案之和，即答案。

4.3.4.3 构造普通生成函数
第 1 种物品的生成函数为 $(1 + x^1 + x^2 + \cdots + x^{a_1})$， 第 n 种物品的生成函数为 $(1 + x^1 + x^2 + \cdots + x^{a_n})$。

即

$$(1 + x^1 + x^2 + \cdots + x^{a_1})(1 + x^1 + x^2 + \cdots + x^{a_2}) \cdots (1 + x^1 + x^2 + \cdots + x^{a_n})$$

求 x^m 的系数。

注意： 指数即物品个数，系数即组合数。

4.3.4.4 例如：
有三种物品，分别有 3、2、1 个，问取 4 个物品的组合数？ 枚举的话，有 {AAAB, AAAC, AABB, AABC, ABBC}，5 个方案。

构造

$$(1 + x + x^2 + x^3)(1 + x + x^2)(1 + x)$$

逐步展开：

$$\begin{aligned}
&= (1 + x + x^2 + x^3)(1 + x + x^2)(1 + x) \\
&= (1 + x + x^2 + x^3 + x^2 + x^3 + x^4 + x^3 + x^4 + x^5)(1 + x) \\
&= (1 + 2x + 3x^2 + 3x^3 + 3x^4 + 2x^5 + x^6)(1 + x) \\
&= 1 + 3x + 5x^2 + 6x^3 + 5x^4 + 3x^5 + x^6
\end{aligned}$$

x^4 的系数为 5，即答案。
4.3.4.5 HDU - 1085 Holding Bin-Laden Captive!
面值为 1, 2, 5 的硬币分别有 a_1, a_2, a_3 枚， 问用这些硬币不能组成的最小面值是多少?

4.3.4.6 思路
构造生成函数：

$$(1 + x^1 + x^2 + \cdots + x^{a_1}) \times (1 + x^2 + x^4 + \cdots + x^{2a_2}) \times (1 + x^5 + x^{10} + \cdots + x^{5a_3})$$

从小到大遍历系数，为 0 的那一项就是答案。

4.3.4.7 例如：
1 分有 1 枚，2 分有 1 枚，5 分有 1 枚：

$$\begin{aligned}
&= (1 + x^2 + x^1 + x^3)(1 + x^5) \\
&= (1 + x^1 + x^2 + x^3)(1 + x^5) \\
&= (1 + x^5 + x^1 + x^6 + x^2 + x^7 + x^3 + x^8) \\
&= 1 + x^1 + x^2 + x^3 + x^5 + x^6 + x^7 + x^8
\end{aligned}$$

最小不能组成的面值是 4。
4.3.4.8 食物
在许多不同种类的食物中选出 n 个，每种食物的限制如下：

1. 承德汉堡：偶数个
2. 可乐：0 个或 1 个
3. 鸡腿：0 个，1 个或 2 个
4. 蜜桃多：奇数个
5. 鸡块：4 的倍数个
6. 包子：0 个，1 个，2 个或 3 个
7. 土豆片炒肉：不超过一个。
8. 面包：3 的倍数个

每种食物都是以「个」为单位，只要总数加起来是 n 就算一种方案。对于给出的 n 你需要计算出方案数，对 10007 取模。

这是一道经典的生成函数题。对于一种食物，我们可以设 a_n 表示这种食物选 n 个的方案数，并求出它的生成函数。而两种食物一共选 n 个的方案数的生成函数，就是它们生成函数的卷积。多种食物选 n 个的方案数的生成函数也是它们生成函数的卷积。

在理解了方案数可以用卷积表示以后，我们就可以构造生成函数（标号对应题目中食物的标号）：

1. $\sum_{n \ge 0} x^{2n} = \frac{1}{1-x^2}$。
2. $1 + x$。
3. $1 + x + x^2 = \frac{1-x^3}{1-x}$。

4. $\sum_{n\geq 0} x^{2n+1} = \sum_{n\geq 0} x^n - \sum_{n\geq 0} x^{2n} = \frac{x}{1-x^2}$。
5. $\sum_{n\geq 0} x^{4n} = \frac{1}{1-x^4}$。
6. $1 + x + x^2 + x^3 = \frac{1-x^4}{1-x}$。
7. $1 + x$。
8. $\sum_{n\geq 0} x^{3n} = \frac{1}{1-x^3}$。

那么全部乘起来，得到答案的生成函数：

$$F(x) = \frac{(1+x)(1-x^3)x(1-x^4)(1+x)}{(1-x^2)(1-x)(1-x^2)(1-x^4)(1-x)(1-x^3)} = \frac{x}{(1-x)^4}$$

广义二项式定理

$$\frac{1}{(1-x)^n} = \sum_{i=0}^{\infty} C_{n+i-1}^i x^i$$

然后将它转化为展开形式（使用广义二项式定理）：

$$\begin{aligned} F(x) &= x\sum_{i=0}^{\infty}(C_{4+i-1}^i)x^i \\ &= \sum_{i=0}^{\infty}(C_{4+i-1}^i)x^{i+1} \\ &= \sum_{k=1}^{\infty}(C_{k+2}^{k-1})x^k \\ &= \sum_{k=1}^{\infty}(C_{k+2}^3)x^k \end{aligned}$$

因此答案为

$$C_{n+2}^3$$

4.3.5 指数生成函数
指数生成函数：

$$F(x) = \sum_{n\geq 0} a_n \frac{x^n}{n!}$$

序列 $< 1, 1, 1, \cdots >$ 的指数生成函数是

$$1 + \frac{x}{1!} + \frac{x^2}{2!} + \frac{x^3}{3!} + \cdots = \sum_{n\geq 0} \frac{x^n}{n!} = e^x$$

序列 $< 1, p, p^2, \cdots >$ 的指数生成函数是

$$1 + p\frac{x}{1!} + p^2\frac{x^2}{2!} + p^3\frac{x^3}{3!} + \cdots = \sum_{n\geq 0} p^n\frac{x^n}{n!} = e^{px}$$

4.3.5.1 基本运算
加减运算

$$F(x) \pm G(x) = \sum_{i\geq 0} a_i\frac{x^i}{i!} \pm \sum_{j\geq 0} b_j\frac{x^j}{j!} = \sum_{n\geq 0}(a_n \pm b_n)\frac{x^n}{n!}$$

因此 $F(x) \pm G(x)$ 是序列 $< a_n \pm b_n >$ 的指数生成函数。

乘法运算（卷积）

$$F(x)G(x) = \sum_{i\geq 0} a_i\frac{x^i}{i!}\sum_{j\geq 0} b_j\frac{x^j}{j!} = \sum_{n\geq 0} x^n\sum_{i=0}^{n} a_i b_{n-i}\frac{1}{i!(n-i)!} = \sum_{n\geq 0}\frac{x^n}{n!}\sum_{i=0}^{n}\frac{n!}{i!(n-i)!}a_i b_{n-i} = \sum_{n\geq 0}\frac{x^n}{n!}\sum_{i=0}^{n} C_n^i a_i b_{n-i}$$

因此 $F(x)G(x)$ 是序列 $< \sum_{i=0}^{n} C_n^i a_i b_{n-i} >$ 的指数生成函数。
4.3.5.2 封闭形式
我们同样考虑指数生成函数的封闭形式。

序列 $\langle 1, 1, 1, \cdots \rangle$ 的指数生成函数是：

$$\hat{F}(x) = \sum_{n\geq 0}\frac{x^n}{n!} = \mathrm{e}^x$$

因为你将 e^x 在 $x = 0$ 处泰勒展开就得到了它的无穷级数形式。

类似地，等比数列 $\langle 1, p, p^2, \cdots \rangle$ 的指数生成函数是：

$$\hat{F}(x)=\sum_{n\geq 0}\frac{p^n x^n}{n!}=\mathrm{e}^{px}$$

4.3.5.3 指数生成函数可以用来解决多重集排列数问题。
HDU - 1521 排列组合 题意：有 n 种物品，每种物品有 a_i 个，问取 m 个物品的排列数？

多重集排列数 设从每种物品中取 b_i 个，$0\leq b_i\leq a_i$，$m=\sum_{i=1}^{n}b_i$，对于一组选定的 b_i 进行排列的方案数为 $\frac{m!}{b_1!b_2!\cdots b_n!}$。若 m 个物品互不相同，其排列数为 $m!$，分母就是对每种相同物品的排列数去重。 例如，取 3 个 A、1 个 B 的排列数为 $\frac{4!}{3!1!}=\frac{24}{6}=4$，即 {AAAB, AABA, ABAA, BAAA}。

取 2 个 A、2 个 B 的排列数为 $\frac{4!}{2!2!}=\frac{24}{4}=6$，即 {AABB, ABAB, ABBA, BAAB. BABA, BBAA}。 那么，所有满足 $b_1+b_2+\cdots+b_n=m$ 的排列数之和，即答案。

构造指数生成函数 第 1 种物品的生成函数为 $\left(1+\frac{x^1}{1!}+\frac{x^2}{2!}+\cdots+\frac{x^{a_1}}{a_1!}\right)$，第 n 种物品的生成函数为 $\left(1+\frac{x^1}{1!}+\frac{x^2}{2!}+\cdots+\frac{x^{a_n}}{a_n!}\right)$。即 $\left(1+\frac{x^1}{1!}+\frac{x^2}{2!}+\cdots+\frac{x^{a_1}}{a_1!}\right)\left(1+\frac{x^1}{1!}+\frac{x^2}{2!}+\cdots+\frac{x^{a_2}}{a_2!}\right)\cdots\left(1+\frac{x^1}{1!}+\frac{x^2}{2!}+\cdots+\frac{x^{a_n}}{a_n!}\right)$，求 $\frac{x^m}{m!}$ 的系数。

做乘法，$\frac{x^{b_1}}{b_1!}\times\frac{x^{b_2}}{b_2!}\times\cdots\times\frac{x^{b_n}}{b_n!}=\frac{x^{b_1+b_2+\cdots+b_n}}{b_1!b_2!\cdots b_n!}=\frac{x^m}{b_1!b_2!\cdots b_n!}=\frac{m!}{b_1!b_2!\cdots b_n!}\cdot\frac{x^m}{m!}$。做卷积，所有满足 $b_1+b_2+\cdots+b_n=m$ 的项的系数之和，再乘以 $m!$，即答案。

4.3.6 一点小结论（前已述及）
(1) 序列 a 的普通生成函数：$F(x)=\sum a_n x^n$

(2) 序列 a 的指数生成函数：$F(x)=\sum a_n\frac{x^n}{n!}$

泰勒展开式

普通生成函数：

$$\frac{1}{1-x}=1+x+x^2+x^3+\cdots=\sum_{n=0}^{\infty}x^n$$

$$\frac{1}{1-x^2}=1+x^2+x^4+\cdots$$

$$\frac{1}{1-x^3}=1+x^3+x^6+\cdots$$

$$\frac{1}{(1-x)^2}=1+2x+3x^2+\cdots$$

指数生成函数：

$$e^x=1+\frac{x^1}{1!}+\frac{x^2}{2!}+\frac{x^3}{3!}+\cdots=\sum_{n=0}^{\infty}\frac{x^n}{n!}$$

$$e^{-x}=1-\frac{x^1}{1!}+\frac{x^2}{2!}-\frac{x^3}{3!}+\cdots$$

$$\frac{e^x+e^{-x}}{2}=1+\frac{x^2}{2!}+\frac{x^4}{4!}+\cdots$$

$$\frac{e^x-e^{-x}}{2}=x+\frac{x^3}{3!}+\frac{x^5}{5!}+\cdots$$

有穷序列的生成函数

$$1+x+x^2=\frac{1-x^3}{1-x}$$

$$1+x+x^2+x^3=\frac{1-x^4}{1-x}$$

广义二项式定理

$$\frac{1}{(1-x)^n}=\sum_{i=0}^{\infty}C_{n+i-1}^{i}x^i$$

证明：

二项式定理：

$$(1+x)^n=\sum_{i=0}^{n}C_n^ix^i$$

⑴ 扩展域：

$$(1+x)^n=\sum_{i=0}^{\infty}C_n^ix^i,\quad \text{当}\quad i>n\quad \text{时}\quad C_n^i=0$$

⑵ 扩展指数为负数：

$$C_{-n}^i=(-n)(-n-1)\cdots(-n-i+1)$$

$$=(-1)^i\cdot\frac{n(n+1)\cdots(n+i-1)}{i!}=(-1)^iC_{n+i-1}^i$$

$$(1+x)^{-n}=\sum_{i=0}^{\infty}C_{-n}^ix^i$$

$$=\sum_{i=0}^{\infty}(-1)^iC_{n+i-1}^ix^i$$

⑶ 括号内的加号变减号：

$$(1-x)^{-n}=\sum_{i=0}^{\infty}(-1)^iC_{n+i-1}^i(-x)^i$$

$$=\sum_{i=0}^{\infty}C_{n+i-1}^ix^i$$

证毕。

5 变换与多项式

5.1 数论/数论笔记部分/FFT 笔记

5.1.1 FFT

多项式的乘法

1. 系数乘法

$A(x) = a_0 + a_1x + a_2x^2 + \cdots + a_{n-1}x^{n-1}$
$B(x) = b_0 + b_1x + b_2x^2 + \cdots + b_{n-1}x^{n-1}$
$C(x) = c_0 + c_1x + c_2x^2 + \cdots + c_{2n-2}x^{2n-2}$

2. 点值乘法

给定 n 个点可以确定 $n-1$ 次函数曲线的系数点值和系数存在映射关系，可以相互转化



(1) 取 $x_0, x_1, x_2, \cdots x_{2n-2}$ 分别代入，求点值
$A(x)$: $y_0, y_1, y_2, \cdots, y_{2n-2}$
$B(x)$: $y'_0, y'_1, y'_2, \cdots, y'_{2n-2}$
(2) 相乘 $C(x)$: $y_0y'_0, y_1y'_1, y_2y'_2, \cdots, y_{2n-2}y'_{2n-2}$
(3) 插值 $C(x)$: $c_0, c_1, c_2, \cdots, c_{2n-2}$

快速傅里叶变换(FFT)

可以把求点值和求系数均优化为 $O(nlogn)$，但是，快速傅里叶变换需要代入复数值，借助**复数运算**才能实现分治递归

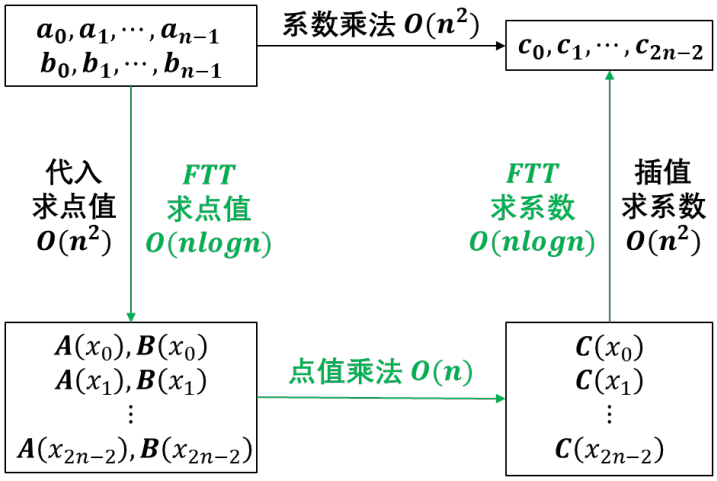


图 2　本地图片

复数

定义：$z = x + iy$，其中 $i^2 = -1$
加法：$z_1 + z_2 = (x_1 + x_2) + i(y_1 + y_2)$
减法：$z_1 - z_2 = (x_1 - x_2) + i(y_1 - y_2)$
乘法：$z_1 * z_2 = (x_1x_2 - y_1y_2) + i(x_1y_2 + x_2y_1)$

单位圆（复平面上圆心在原点且半径为 1 的圆）

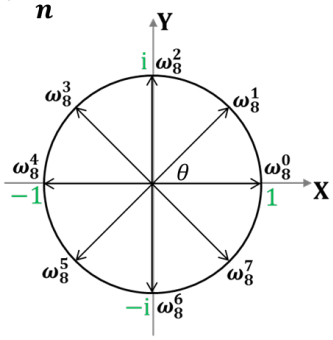
圆上的点：$z = cos\theta + i\ sin\theta$ $(0 \le \theta < 2\pi)$

方程 $z^n = 1$ 有 n 个复数根(圆的 n 个等分点)

单位根 $\boldsymbol{\omega_n^k = cos\frac{2\pi k}{n} + i\ sin\frac{2\pi k}{n}}$

重要性质（注意 $n = 2^{b \in \mathbb{N}}$）

1. 指数性 $\omega_n^k\omega_n^m = \omega_n^{k+m}$
2. 周期性 $\omega_n^{k+n} = \omega_n^k$
3. 对称性 $\omega_n^{k+\frac{n}{2}} = -\omega_n^k$
4. 折半性 $\omega_n^{2k} = \omega_{n/2}^k$



由系数求点值（正变换）

设 $A(x)$ 的系数为 $(a_0, a_1, a_2, \cdots, a_{n-1})$

则 $A(x) = a_0 + a_1x + a_2x^2 + a_3x^3 + \cdots + a_{n-1}x^{n-1}$
$= (a_0 + a_2x^2 + \cdots + a_{n-2}x^{n-2})$ ← 偶项
$+(a_1 + a_3x^2 + \cdots + a_{n-1}x^{n-2})x$ ← 奇项

设 $A_1(x) = a_0 + a_2x + a_4x^2 + \cdots + a_{n-2}x^{\frac{n}{2}-1}$
$A_2(x) = a_1 + a_3x + a_5x^2 + \cdots + a_{n-1}x^{\frac{n}{2}-1}$

则 $A(x) = A_1(x^2) + A_2(x^2)x$

将 $\omega_n^k (k < \frac{n}{2})$ 代入得

$$A(\omega_n^k) = A_1(\omega_n^{2k}) + A_2(\omega_n^{2k})\omega_n^k$$
$$= A_1\left(\omega_{\frac{n}{2}}^k\right) + A_2\left(\omega_{\frac{n}{2}}^k\right)\omega_n^k$$

将 $\omega_n^{k+\frac{n}{2}}$ 代入得

$$A\left(\omega_n^{k+\frac{n}{2}}\right) = A_1(\omega_n^{2k+n}) + A_2(\omega_n^{2k+n})\omega_n^{k+\frac{n}{2}}$$
$$= A_1\left(\omega_{\frac{n}{2}}^k\right) - A_2\left(\omega_{\frac{n}{2}}^k\right)\omega_n^k$$

图 3　本地图片

$A(x)$ 的系数为 $(a_0, a_1, a_2, \cdots, a_{n-1})$
$A_1(x)$ 的系数为 $(a_0, a_2, a_4, \cdots, a_{n-2})$
$A_2(x)$ 的系数为 $(a_1, a_3, a_5, \cdots, a_{n-1})$

$\omega_n^1 = \cos\frac{2\pi}{n} + i\sin\frac{2\pi}{n}$，$\omega_n^k = (\omega_n^1)^k$

$k < \frac{n}{2}$ 时，$A(\omega_n^k) = A_1\left(\omega_{\frac{n}{2}}^k\right) + A_2\left(\omega_{\frac{n}{2}}^k\right)\omega_n^k$

后一半，$A\left(\omega_n^{k+\frac{n}{2}}\right) = A_1\left(\omega_{\frac{n}{2}}^k\right) - A_2\left(\omega_{\frac{n}{2}}^k\right)\omega_n^k$

A数组初值：实部存系数，虚部存0。最终返回多项式的点值
每个结点向下劈叉，都新开两个数组A_1, A_2。自底向上合并

一共 $logn$ 层，每层计算量为 $O(n)$，时间复杂度为 $O(nlogn)$

$$A(\omega_8^1) = (a_0 - a_4\omega_2^0) + (a_2 - a_6\omega_2^0)\omega_4^1 + [(a_1 - a_5\omega_2^0) + (a_3 - a_7\omega_2^0)\omega_4^1]\omega_8^1$$

$$A(\omega_8^5) = (a_0 - a_4\omega_2^0) + (a_2 - a_6\omega_2^0)\omega_4^1 - [(a_1 - a_5\omega_2^0) + (a_3 - a_7\omega_2^0)\omega_4^1]\omega_8^1$$

```
void FFT(complex A[],int n){
 if(n==1) return;
 complex A1[n/2],A2[n/2];
 for(int i=0;i<n/2;++i)
   A1[i]=A[i*2], A2[i]=A[i*2+1];
 FFT(A1,n/2); FFT(A2,n/2);
 complex w1({cos(2*PI/n),sin(2*PI/n)});
 complex wk({1,0});
 for(int i=0;i<n/2;++i){
   A[i]=A1[i]+A2[i]*wk;
   A[i+n/2]=A1[i]-A2[i]*wk; wk=wk*w1;}
}
```

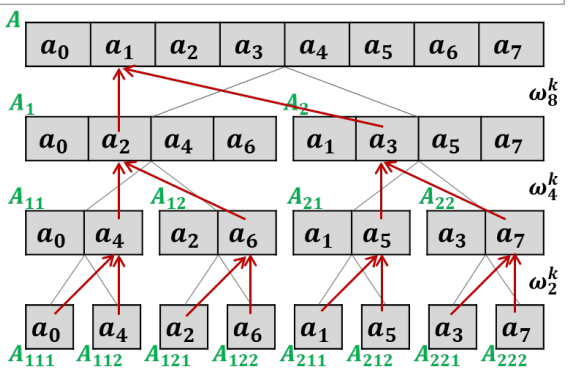


图 4　本地图片

由点值求系数（逆变换）

设多项式 $A(x) = a_0 + a_1x + a_2x^2 + \cdots + a_{n-1}x^{n-1}$

代入 $\omega_n^0, \omega_n^1, \omega_n^2, \cdots, \omega_n^{n-1}$ 得到的点值为$(y_0, y_1, y_2, \cdots, y_{n-1})$

其中 $y_i = \sum_{j=0}^{n-1} a_j(\omega_n^i)^j$

构造多项式 $B(x) = y_0 + y_1x + y_2x^2 + \cdots + y_{n-1}x^{n-1}$

把 n 个单位根的**倒数** $\omega_n^0, \omega_n^{-1}, \omega_n^{-2}, \cdots, \omega_n^{-(n-1)}$ 代入 $B(x)$
得到的 n 个新点值，设为 $(z_0, z_1, z_2, \cdots, z_{n-1})$

$$z_k = \sum_{i=0}^{n-1} y_i(\omega_n^{-k})^i = \sum_{i=0}^{n-1}\sum_{j=0}^{n-1} a_j(\omega_n^i)^j(\omega_n^{-k})^i = \sum_{j=0}^{n-1} a_j \sum_{i=0}^{n-1}(\omega_n^{j-k})^i$$

当 $j = k$ 时，内层的和式等于 n

当 $j \neq k$ 时，$\frac{\left(\omega_n^{j-k}\right)^n - 1}{\omega_n^{j-k} - 1} = \frac{(\omega_n^n)^{j-k} - 1}{\omega_n^{j-k} - 1} = 0$

所以 $z_k = na_k$，即 $a_k = \frac{z_k}{n}$　（单位根的神奇！）

$$\omega_n^k = cos\theta + isin\theta$$

$$\frac{1}{\omega_n^k} = \frac{1}{cos\theta + isin\theta} = cos\theta - isin\theta = \cos(-\theta) + isin(-\theta) = \omega_n^{-k}$$

逆变换代入单位根的**倒数** 等价于虚部变负号

```
void IFFT(complex A[],int n){
  if(n==1) return;
  complex A1[n/2],A2[n/2];
  for(int i=0;i<n/2;++i)
    A1[i]=A[i*2], A2[i]=A[i*2+1];
  FFT(A1,n/2); FFT(A2,n/2);
  complex w1({cos(2*PI/n),-sin(2*PI/n)});
  complex wk({1,0});
  for(int i=0;i<n/2;++i){
    A[i]=A1[i]+A2[i]*wk;
    A[i+n/2]=A1[i]-A2[i]*wk; wk=wk*w1;}
}
```

图 5　本地图片

Luogu P3803 【模板】多项式乘法（FFT）

$\omega_n^1 = cos\frac{2\pi}{n} + i\,sin\frac{2\pi}{n}$，$\omega_n^k = (\omega_n^1)^k$

$k < \frac{n}{2}$时，$A(\omega_n^k) = A_1\left(\omega_{\frac{n}{2}}^k\right) + A_2\left(\omega_{\frac{n}{2}}^k\right)\omega_n^k$

后一半，$A\left(\omega_n^{k+\frac{n}{2}}\right) = A_1\left(\omega_{\frac{n}{2}}^k\right) - A_2\left(\omega_{\frac{n}{2}}^k\right)\omega_n^k$

逆变换，单位根的虚部变负号，结果再除以 n

几个细节

1. 增加一个选择参数，统一板子

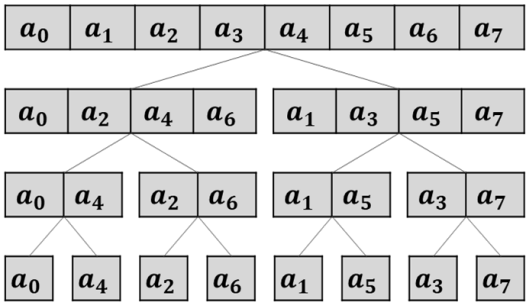
2. 确定 m 次多项式的系数，至少取 $m+1$ 个点
   选取 n 个单位根，必须满足：$m < n = 2^{b\in\mathbb{N}}$
   例如，$m = \{4,5,6,7\}$ 时，取 $n = 8$
   本题 $n, m \le 10^6$，所以数组开 4×10^6

3. 浮点数计算有误差，例如 $[5.999998 + 0.5] = 6$

```
const int N=4e6;
const double PI=acos(-1);
complex A[N],B[N];

void FFT(complex A[],int n,int op){
  if(n==1) return;
  complex A1[n/2],A2[n/2];
  for(int i=0;i<n/2;++i)
    A1[i]=A[i*2], A2[i]=A[i*2+1];
  FFT(A1,n/2,op); FFT(A2,n/2,op);
  complex w1({cos(2*PI/n),sin(2*PI/n)*op});
  complex wk({1,0});
  for(int i=0;i<n/2;++i){
    A[i]=A1[i]+A2[i]*wk;
    A[i+n/2]=A1[i]-A2[i]*wk; wk=wk*w1;}
}
int main(){
  int n,m; scanf("%d%d",&n,&m);
  for(int i=0;i<=n;++i)scanf("%lf",&A[i]);
  for(int i=0;i<=m;++i)scanf("%lf",&B[i]);
  for(m=n+m,n=1;n<=m;n<<=1); //求n
  FFT(A,n,1); FFT(B,n,1); //求点值
  for(int i=0;i<n;++i)A[i]=A[i]*B[i];//乘积
  FFT(A,n,-1); //求系数
  for(int i=0;i<=m;++i)
    printf("%d ",(int)((A[i].real())/n+0.5));
}
```

图 6　本地图片



原序列： 0 1 2 3 4 5 6 7
000 001 010 011 100 101 110 111

后序列： 0 4 2 6 1 5 3 7
000 100 010 110 001 101 011 111

规律：每个位置的元素下标都做了**二进制翻转**，这个变换称为**位逆序变换**（也称**蝴蝶变换**）

先利用**位逆序变换**得到后序列，然后自底向上直接合并就行了，把递归版改为迭代版。

位逆序变换可以 $O(n)$ 递推实现

设 $n = 2^k$，k 表示二进制数的长度
设 $R[x]$ 表示长度为 k 的二进制数 x 翻转后的数
我们要求出 $R[0], R[1], \cdots, R[n-1]$，初值 $R[0] = 0$

```
void change(complex A[], int n){
  for(int i=0; i<n; ++i)
    R[i] = R[i/2]/2 + ((i&1)?n/2:0);
  for(int i=0; i<n; ++i)
    if(i<R[i]) swap(A[i],A[R[i]]);
}
```

$$R[x] = \frac{R[\frac{x}{2}]}{2} + [x\&1] \times \frac{n}{2}$$

例：设 $n = (100000)_2$，$k = 5$，$x = (11001)_2$
1. x 右移一位 $(01100)_2$，$R[(01100)_2] = (00110)_2$，再右移一位 $(00011)_2$
2. x 个位是 1，所以最高位要加上 $(10000)_2 = \frac{n}{2}$

图 7　本地图片

```
void FFT(complex A[],int n,int op){
 if(n==1) return;
 complex A1[n/2],A2[n/2];
 for(int i=0;i<n/2;++i)
   A1[i]=A[i*2], A2[i]=A[i*2+1];
 FFT(A1,n/2,op); FFT(A2,n/2,op);
 complex w1({cos(2*PI/n),sin(2*PI/n)*op});
 complex wk({1,0});
 for(int i=0;i<n/2;++i){
   A[i]=A1[i]+A2[i]*wk;
   A[i+n/2]=A1[i]-A2[i]*wk; wk=wk*w1;
 }
}
```

2.1s

递归合并

```
void FFT(complex A[],int n,int op){
 change(A, n); //位逆序变换
 for(int m=2; m<=n; m<<=1){ //枚举块宽
   complex w1({cos(2*PI/m),sin(2*PI/m)*op});
   for(int i=0; i<n; i+=m){ //枚举块数
     complex wk({1,0});
     for(int j=0; j<m/2; ++j){ //枚举半块
       complex x=A[i+j], y=A[i+j+m/2]*wk;
       A[i+j]=x+y; A[i+j+m/2]=x-y; wk=wk*w1;
     }
   }
 }
}
```

1.5s

按层合并

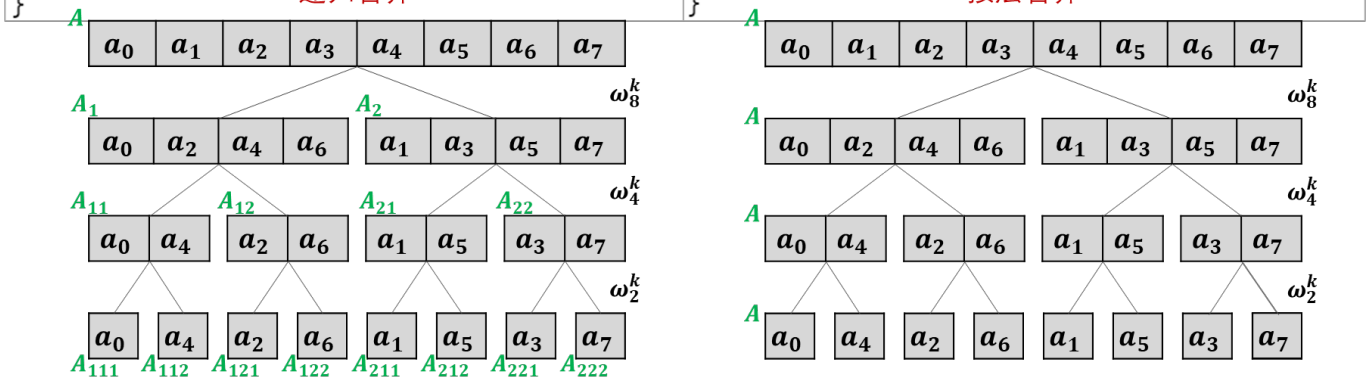


图 8　本地图片

5.2 数论/数论笔记部分/数论笔记(sosdp&fmt&fwt)

5.2.1 sosdp(高维前缀和)

我们要求 $dp[i]$ 的所有子集 $j \subseteq i$ 的 $A[j]$ 之和。

考虑二维前缀和的另外一种求法 想象一个二维矩阵，我们怎么用“笨办法”求前缀和？ 我们可以分两步走：

先只处理行（x 轴）： 对每一行单独做 1D 前缀和。

再只处理列（y 轴）： 基于第一步的结果，对每一列单独做 1D 前缀和。

5.2.1.1 推广到 N 维（比特位）

在 SOS DP 中，我们把一个数字的二进制表示看作维度。比如数字 $5(101_2)$，可以看作是一个 3 维坐标 $(1,0,1)$。求“子集和”（即 i 的所有子集 $j \subseteq i$ 的 $A[j]$ 之和），本质上就是在求这个 N 维超立方体的“高维前缀和”。

我们只需要：

1. 先处理第 0 位（维度 0）：把所有第 0 位是 0 的数，累加到第 0 位是 1 的数上。
2. 再处理第 1 位（维度 1）：基于上一步，把所有第 1 位是 0 的数，累加到第 1 位是 1 的数上。
3. …
4. 直到处理完第 $N-1$ 位。

如果泥玩过 2048 的话，应该能理解这个方法

```
// dp[mask] 初始存的是单点的值 A[mask]
// n 是二进制的位数（维度）

for (int j = 0; j < n; ++j) { // 枚举每一个维度（每一位）
    for (int i = 0; i < (1 << n); ++i) { // 枚举所有状态
        if ((i >> j) & 1) { // 如果当前状态 i 在第 j 维上是 1
            // 那么它包含“第 j 维是 0”的那个状态 (i ^ (1 << j))
            dp[i] += dp[i ^ (1 << j)];
        }
    }
}
```

精细实现可以做到$O(n*2^{()n-1})$ 见杂项 tirck

我们追踪一下 dp[7(111)]

假设 $N=3$，我们要算 $dp[111_2]$ (即 $dp[7]$)。

1. 初始： $dp[7]$ 只包含 $A[7]$。
2. **j=0** (处理第 0 位)： $dp[...1] += dp[...0]$。 此时 $dp[111]$ 加上了 $dp[110]$。 意义：现在 $dp[111]$ 包含了 $\{111,110\}$ 的和。

3. **j=1** (处理第 1 位)： $dp[..1.] += dp[..0.]$。 此时 $dp[111]$ 加上了 $dp[101]$。 注意： 这里的 $dp[101]$ 在上一轮已经加上了 $dp[100]$。 意义：现在 $dp[111]$ 包含了 $\{111, 110, 101, 100\}$ 的和。
4. **j=2** (处理第 2 位)： $dp[1..] += dp[0..]$。 此时 $dp[111]$ 加上了 $dp[011]$。 注意： $dp[011]$ 在前两轮已经包含了 $\{011, 010, 001, 000\}$。 意义：现在 $dp[111]$ 包含了所有 8 个子集。

5.2.1.2 超集和
换个符号即可

```
for (int j = 0; j < n; ++j) {
    for (int i = 0; i < (1 << n); ++i) {
        if (!((i >> j) & 1)) { // 如果第 j 位是 0
            // 它的超集包括第 j 位是 1 的那个状态
            dp[i] += dp[i ^ (1 << j)];
        }
    }
}
```

如果做高维差分 反着做前缀和即可

5.2.1.3 例题：luogu P5495 【模板】Dirichlet 前缀和
唯一分解定理：$N = \prod p_i^{c_i}$，其中 p_i 是质数，c_i 是次数。

我们把 p_i 看作维度，c_i 看作第 p_i 维的值。 $e.g.\ 12 = 2^2 \times 3^1$，那么 12 可以看作是 $(2, 1, 0...)$。

考虑先枚举 p_i， 再在每个维度上做偏序关系转移即可
5.2.1.4 例题：CF449D
考虑答案是 st 的超集的方案数是好求的：

我们首先统计 `a[i]`是 `st` 的超集的个数 这里用下 sosdp 求

答案即为$2^{cnt[st]} - 1$

现在我们有至少 考虑恰好的关系

用一下超集反演：

$$f(S) = \sum_{S \subseteq T} g(T) \Leftrightarrow g(S) = \sum_{S \subseteq T} (-1)^{|T|-|S|} f(T)$$

我们知道

$$f(0) = \sum_{S \subseteq T} g(T)$$

这里的 f 函数含义是 答案是 0 的超集的方案数 右边是 对所有 0 的超集 恰好为 t 的方案数

所以

$$g(0) = \sum_{S \subseteq T} (-1)^{|T|} f(T)$$

其实这是简单容斥

容斥求解即可
5.2.2 FMT(快速莫比乌斯变换)
回忆多项式乘法

$$C_k = \sum_{i+j=k} A_i \times B_j$$

考虑把下标 换成按位或

$$C_k = \sum_{i \vee j=k} A_i \times B_j$$

你如果把下标看成集合意义下的状压的话 可能比较好理解一点 e.g. $i \vee j = k$ 就是说 i 和 j 的并集是 k

这就是所谓的“按位或卷积”

我们知道,FFT 的原理是把系数乘法换成点值乘法，以达到加速的目的

or 卷积也能这样加速

定义：

$$FMT(A)[k]=\sum_{i\subseteq k}A[i]$$

注意这是子集和的形式

考虑：

$$FMT(A)[k]\times FMT(B)[k]=\left(\sum_{i\subseteq k}A[i]\right)\times\left(\sum_{j\subseteq k}B[j]\right)$$

$$=\sum_{i\subseteq k}\sum_{j\subseteq k}A[i]B[j]$$

注意条件：如果 i 是 k 的子集，且 j 也是 k 的子集，那么 $i\vee j$ 一定也是 k 的子集。 所以上面那坨式子其实等于：

$$\sum_{(i\vee j)\subseteq k}A[i]B[j]$$

按定义，我们知道：

$$FMT(C)[k]=\sum_{i\subseteq k}C[i]=\sum_{(i\vee j)\subseteq k}A[i]B[j]$$

所以：

$$FMT(A)[k]\times FMT(B)[k]=FMT(C)[k]$$

所以我们可以把多项式乘法变成 FMT 乘法，然后再 $IFMT$ 回去。

这里的 or 卷积的 FMT 就是高维前缀和，IMFT 就是高维前缀差分

同样的我们有 and 卷积

$C_k=\sum_{i\wedge j=k}A_i\times B_j$

推导基本从上，恕我从略，只要把高维前缀和变成高维后缀和，高维前缀差分变成高维后缀差分即可

5.2.3 FWT(快速沃尔什变换)

FWT 除了可以处理 and/or 卷积，还可以处理异或卷积

异或卷积：

$$C_k=\sum_{i\oplus j=k}A_i\times B_j$$

下面详细推导一下 xor 卷积的 FWT

构造：

$$FWT(A)_k=\sum_{i=0}^{2^n-1}(-1)^{|\ i\cap k\ |}A_i$$

其中 $|\ x\ |$ 表示 x 二进制表示中 1 的个数（popcount）。 $|\ i\cap k\ |$ 表示 i 和 k 按位与之后，包含 1 的个数。 $(-1)^{|\ i\cap k\ |}$ 实际上是在检测 i 和 k 的“相关性”奇偶。

我们要证明：$FWT(C)_k=FWT(A)_k\cdot FWT(B)_k$。 推导左边 (LHS)：根据定义，把 $C_p=\sum_{i\oplus j=p}A_iB_j$ 代入：

$$FWT(C)_k=\sum_p(-1)^{|\ p\cap k\ |}C_p$$

$$=\sum_p(-1)^{|\ p\cap k\ |}\left(\sum_{i\oplus j=p}A_iB_j\right)$$

由于 $p=i\oplus j$，我们可以把求和号展开，直接枚举 i 和 j：

$$FWT(C)_k=\sum_i\sum_j(-1)^{|\ (i\oplus j)\cap k\ |}A_iB_j$$

关键性质：

$$|\ (i\oplus j)\cap k\ |\equiv|\ i\cap k\ |+|\ j\cap k\ |\pmod 2$$

解释：i 和 j 在第 m 位异或为 1，当且仅当它们在该位上一个为 0 一个为 1。与 k 做 AND 后，只有当 k 在该位也是 1 时才会产生贡献。这一位的贡献要么是 $1+0=1$，要么是 $0+1=1$，奇偶性都和加法一致。因为 $(-1)^x$ 只关心 x 的奇偶性，所以：

$$(-1)^{|\ (i\oplus j)\cap k\ |}=(-1)^{|\ i\cap k\ |+|\ j\cap k\ |}=(-1)^{|\ i\cap k\ |}\cdot(-1)^{|\ j\cap k\ |}$$

代回原式：

$$\begin{aligned}FWT(C)_k&=\sum_i\sum_j\left((-1)^{|\ i\cap k\ |}\cdot(-1)^{|\ j\cap k\ |}\right)A_iB_j\\&=\left(\sum_i(-1)^{|\ i\cap k\ |}A_i\right)\cdot\left(\sum_j(-1)^{|\ j\cap k\ |}B_j\right)\\&=FWT(A)_k\cdot FWT(B)_k\end{aligned}$$

得证。

现在我们推导 FWT 的分治算法

假设数组 A 的长度为 2^n。我们将 A 分成前半部分 A_0（下标最高位为 0）和后半部分 A_1（下标最高位为 1）。即：* A_0 包含下标 $0\sim2^{n-1}-1$ * A_1 包含下标 $2^{n-1}\sim2^n-1$

现在我们要计算 $FWT(A)$。对于任意结果下标 k，我们把它写成 $(0,k')$ 或者 $(1,k')$ 的形式，其中 k' 是去掉最高位剩下的部分。
5.2.3.1 情况 1：结果下标最高位为 0 ($k<2^{n-1}$)

$$\begin{aligned}FWT(A)_k&=\sum_{i=0}^{2^n-1}(-1)^{|\ i\cap k\ |}A_i\\&=\sum_{i=0}^{2^{n-1}-1}(-1)^{|\ i\cap k\ |}A_i+\sum_{i=2^{n-1}}^{2^n-1}(-1)^{|\ i\cap k\ |}A_i\end{aligned}$$

- 左半边和 ($\boldsymbol{A_0}$)：i 的最高位是 0，k 的最高位是 0。$i\cap k$ 不涉及最高位。这部分就是 $FWT(A_0)_k$。
- 右半边和 ($\boldsymbol{A_1}$)：i 的最高位是 1，k 的最高位是 0。$i\cap k$ 依然不涉及最高位（因为 $1\ \ 0=0$）。这部分就是 $FWT(A_1)_k$。

结论 1：

$$FWT(A)_{left}=FWT(A_0)+FWT(A_1)$$

5.2.3.2 情况 2：结果下标最高位为 1 ($k\geq2^{n-1}$)
令实际计算的下标为 $k+2^{n-1}$（其中 $k<2^{n-1}$）。

$$FWT(A)_{k+2^{n-1}}=\sum_{i=0}^{2^{n-1}-1}(-1)^{|\ i\cap(k+2^{n-1})\ |}A_i+\sum_{i=2^{n-1}}^{2^n-1}(-1)^{|\ i\cap(k+2^{n-1})\ |}A_i$$

- 左半边和 ($\boldsymbol{A_0}$)：i 最高位 0。$i\cap\left(k+2^{n-1}\right)$ 的最高位是 $0\ \ 1=0$。最高位无贡献。这部分依然是 $FWT(A_0)_k$。
- 右半边和 ($\boldsymbol{A_1}$)：i 最高位 1。$i\cap\left(k+2^{n-1}\right)$ 的最高位是 $1\ \ 1=1$。最高位贡献了 1 个 1。所以 $(-1)^{|\cdots|}$ 会多乘一个 $(-1)^1=-1$。这部分变成了 $-FWT(A_1)_k$。

结论 2：

$$FWT(A)_{right}=FWT(A_0)-FWT(A_1)$$

综合上面的推导，我们得到了递归式：

$$FWT(A)=\text{merge}\ (FWT(A_0),FWT(A_1))$$

其中 merge 操作为：

$$\begin{aligned}\{A'_{left}&=A_{0'}+A_{1'}\\A'_{right}&=A_{0'}-A_{1'}\end{aligned}$$

这正是 FWT 的分治算法。

IFWT 对反解上述过程即可

同理 我们可以推导
5.2.3.3 or 卷积的 FWT

$$FWT(A)_k=\sum_{i\subseteq k}A_i$$

假设数组 A 长度为 2^n。我们将 A 一分为二，同样可以推导 正变换:

$$\begin{aligned}\{A'_{left}&=A'_0\\A'_{right}&=A'_0+A'_1\end{aligned}$$

逆变换:解上面的方程组求 A'_0,A'_1：

$$\{A'_0 = A'_{left}$$
$$A'_1 = A'_{right} - A'_{left}$$

5.2.3.4 and 卷积的 FWT

$$FWT(A)_k = \sum_{k \subseteq i} A_i$$

正变换:

$$\{A'_{left} = A'_0 + A'_1$$
$$A'_{right} = A'_1$$

逆变换:解方程组:

$$\{A'_1 = A'_{right}$$
$$A'_0 = A'_{left} - A'_{right}$$

5.2.3.5 例题： luogu P5387 [Cnoi2019] 人形演舞
观察到游戏是独立的，打表出 sg 函数 sg(x)=x-highbit(x)+1

问题变成：计算方案数，使得 $SG(v_1) \oplus SG(v_2) \oplus ... \oplus SG(v_n) \neq 0$

考虑正难则反 用总方案数 m^n减去 $SG(v_1) \oplus SG(v_2) \oplus ... \oplus SG(v_n) = 0$ 的方案数

我们用一个 cnt 数组 cnt[k]统计 sg=k 的数的个数

答案就是这个数组自卷 v 次的 cnt[0]

> 为啥？ 假设只有两轮游戏，或者只选了两个数。 第一个数选出的 SG 值为 i 的方案数是 $cnt[i]$。 第二个数选出的 SG 值为 j 的方案数是 $cnt[j]$。 现在我想知道：这两个数的异或和 $i \oplus j = k$ 的方案数是多少？根据乘法原理，对于固定的 i 和 j，方案数是 $cnt[i] \times cnt[j]$。我们要找所有满足 $i \oplus j = k$ 的情况，把它们加起来：

$$\text{Total } [k] = \sum_{i \oplus j = k} cnt[i] \times cnt[j]$$

这就是 xor 卷积

先 fwt 一下 然后对点值求 v 次方 最后 ifwt 回去即可

5.3 数论/数论笔记部分/数论笔记(阶,原根与 ntt)

5.3.1 1.1 欧拉定理

欧拉定理：设 $a \in \mathbb{Z}, m \in \mathbb{N}^*$，若 $\gcd(a, m) = 1$，则 $a^{\varphi(m)} \equiv 1 \pmod m$。$\varphi(m)$ 是欧拉函数，表示小于 m 且与 m 互质的正整数个数。

> 证明：考虑模 m 的简化剩余系，即

$$R = \left\{r_1, r_2, \cdots, r_{\varphi(m)}\right\}$$

这个集合包含了所有在 1 到 m 之间且与 m 互质的整数。显然，这个集合里有 $\varphi(m)$ 个元素。 现在，我们把集合 R 里的每一个元素都乘上 a，注意$\gcd(a, m) = 1$，得到一个新的集合：

$$S = \left\{ar_1, ar_2, ..., ar_{\varphi(m)}\right\}$$

注意到 1.S 中的元素都与 m 互质; 2.S 中的元素在模 m 下两两不同(可以用反证法证明)。 既然 S 中有 $\varphi(m)$ 个与 m 互质且模 m 两两不同的数，那么它们在模 m 意义下一定与 R 里的数一一对应。 那么：

$$\prod_{i=1}^{\varphi(m)} (ar_i) \equiv \prod_{i=1}^{\varphi(m)} r_i \pmod m$$

$$a^{\varphi(m)} \prod_{i=1}^{\varphi(m)} r_i \equiv \prod_{i=1}^{\varphi(m)} r_i \pmod m$$

$$a^{\varphi(m)} \equiv 1 \pmod m$$

这边用到的一个厉害的性质是，模 m 的简化剩余系，他在模 m 意义下对乘法（和 m 互质的数 a）是封闭的。

5.3.2 1.2.1 阶，原根

阶：若满足同余式 $a^n \equiv 1 \pmod m$ 的最小正整数解 n 存在，则称 n 为 a 模 m 的阶，记作 $\delta_m(a)$。原根：设 $a \in \mathbb{Z}$，$m \in \mathbb{N}^*$，若 $\delta_m(a) = \varphi(m)$，则称 a 为模 m 的一个原根。

> 不难注意， $\delta_m(a)$ 是一个最小循环节，而$\delta_m(a)$ 一定是 $\varphi(m)$ 的一个约数。

5.3.3 1.2.2 原根数量

如何求解 m 的原根数量？ 因为 a 是 m 的一个原根，所以 $a^1, a^2, a^3, \cdots, a^{\varphi(m)}$ 在取模 m 意义下均不相同（否则就与 $\delta_m(a)$ 的最短循环节定义相矛盾了）。 设t是某个与$\varphi(m)$互质的正整数。 容易得到$t, 2t, \cdots, \varphi(m) \cdot t$在模$\varphi(m)$意义下均不相同。

实际上这个结论是，完全剩余系在模 m 意义下对乘法（乘一个与m互质的数）是封闭的。（证明和上面是一致的）

根据欧拉定理，那么 $a^t, a^{2t}, \cdots, a^{\varphi(m)\cdot t}$ 在取模 m 的意义下也就两两不同。所以 a^t 也是模 m 的一个原根，因为 t 是一个跟 $\varphi(m)$ 互质的数，也就是说满足这样条件的数有 $\varphi(\varphi(m))$ 个。

由于 a^t 的不断乘方（即 $(a^t)^1, (a^t)^2, ...$）也能遍历所有互质同余类，这完全符合原根的定义。所以，a^t 就是 m 的一个新的原根。

5.3.4 1.2.3 原根存在定理

原根存在定理： 一个数 m 存在原根，当且仅当

$$m = 2, 4, p^\alpha, 2p^\alpha$$

其中 p 为奇素数，$\alpha \in \mathbb{N}^*$。

太难了，不想证。 然后我们就能 O(m^(1/4) *logm^2)求出 m 的原根了。

5.3.5 2. NTT

5.3.5.1 2.1 性质

考虑单位根和原根所构造的单位根的相似性，有如下四点性质: 1. 周期性 FFT（复数域）： $\omega_n^n = e^{2\pi i} = 1$。转了一圈回到 1。NTT（模 p 域）： 考虑 NTT 的模数是质数 $p = c \cdot 2^k + 1$ 的形式，考虑g是模p的原根，根据欧拉定理和上述定义，我们知道 $g^{p-1} \equiv 1 \pmod p$。考虑块长$n = 2^t$ 且 $n \mid (p-1)$，令$g_n \equiv g^{\frac{p-1}{n}} \pmod p$（单位根），那么$(g_n)^n \equiv \left(g^{\frac{p-1}{n}}\right)^n \equiv g^{p-1} \equiv 1 \pmod p$

对比 FFT，可以发现此处的2^t正好是块长。2. 互异性 FFT（复数域）： $\omega_n^0, \omega_n^1, ..., \omega_n^{n-1}$ 在复平面上均匀分布，互不相等。这样代入多项式才能得到 n 个独立的点值。 NTT（模 p 域）： $g_n^0, g_n^1, ..., g_n^{n-1}$ 模 p 下互不相同，此处由原根的性质保证。 3. 折半引理 FFT（复数域）： $\omega_{2n}^2 = \omega_n$。这是 FFT 能把偶数项和奇数项拆开递归的核心。 NTT（模 p 域）：注意：

$$g_{2n}^2 \equiv \left(g^{\frac{p-1}{2n}}\right)^2 \equiv g^{\frac{p-1}{n}} \equiv g_n \pmod p$$

4. 求和引理 FFT（复数域）： 对于任意不为 0 的整数 k（且 k 不是 n 的倍数），$\sum_{j=0}^{n-1} \left(\omega_n^k\right)^j = 0$。这是把点值转回系数的关键。 NTT（模 p 域）： 当 $n \nmid k$ 时，$g_n^k \not\equiv 1 \pmod p$，这是一个等比数列求和：

$$\sum_{j=0}^{n-1} \left(g_n^k\right)^j \equiv \frac{1 - \left(g_n^k\right)^n}{1 - g_n^k} \pmod p$$

注意分子为0，分母不为0，这个式子依旧$= 0$。

5.3.5.2 2.2 递归过程

考虑重新手推一下 NTT 的递归过程。 假设我们有一个长度为 n（n 是 2 的幂次）的多项式：

$$A(x) = a_0 + a_1 x + a_2 x^2 + ... + a_{n-1} x^{n-1}$$

我们要把 n 个点 $x = g_n^0, g_n^1, ..., g_n^{n-1}$ 代入进去求值。设：偶数项：$A_0(x) = a_0 + a_2 x + a_4 x^2 + ... + a_{n-2} x^{\frac{n}{2}-1}$ 奇数项：$A_1(x) = a_1 + a_3 x + a_5 x^2 + ... + a_{n-1} x^{\frac{n}{2}-1}$ 于是：

$$A(x) = A_0(x^2) + x \cdot A_1(x^2)$$

带入$x = g_n^k$:

$$A(g_n^k) = A_0\left(\left(g_n^k\right)^2\right) + g_n^k \cdot A_1\left(\left(g_n^k\right)^2\right)$$

注意:

$$\left(g_n^k\right)^2 \equiv \left(g^{k \cdot \frac{p-1}{n}}\right)^2 \equiv g^{k \cdot \frac{p-1}{n/2}} \equiv g_{n/2}^k \pmod p$$

于是:

$$A(g_n^k) \equiv A_0\left(g_{n/2}^k\right) + g_n^k \cdot A_1\left(g_{n/2}^k\right) \pmod p$$

考虑后半段$g_n^{k+n/2}$（此时 $k < n/2$），代入：

$$A\left(g_n^{k+n/2}\right) = A_0\left(\left(g_n^{k+n/2}\right)^2\right) + g_n^{k+n/2} \cdot A_1\left(\left(g_n^{k+n/2}\right)^2\right)$$

注意：

$$\left(g_n^{k+n/2}\right)^2 \equiv g_n^{2k+n} \equiv g_n^{2k} \cdot g_n^n \equiv g_n^{2k} \cdot 1 \equiv \left(g_n^k\right)^2 \equiv g_{n/2}^k \pmod p$$

再次注意：

$$g_n^{k+n/2} \equiv g_n^k \cdot g_n^{n/2} \pmod p$$

其中 $g_n^{n/2}$：

$$g_n^{n/2} \equiv \left(g^{\frac{p-1}{n}}\right)^{\frac{n}{2}} \equiv g^{\frac{p-1}{2}} \pmod p$$

于是：

$$g_n^{k+n/2} \equiv -g_n^k \pmod p$$

最终： 前半段：

$$A(g_n^k) \equiv A_0\left(g_{n/2}^k\right) + g_n^k \cdot A_1\left(g_{n/2}^k\right) \pmod p$$

后半段：

$$A\left(g_n^{k+n/2}\right) \equiv A_0\left(g_{n/2}^k\right) - g_n^k \cdot A_1\left(g_{n/2}^k\right) \pmod p$$

把递归写成递推跟 FFT 是一致的，不再赘述

5.3.5.3 2.3 INTT
假设我们原本的多项式系数是 $a_0, a_1, ..., a_{n-1}$。经过正向 NTT 后，我们得到了 n 个点值 $y_0, y_1, ..., y_{n-1}$。正向 NTT 的方程是这样的：

$$y_k = \sum_{j=0}^{n-1} a_j \left(g_n^k\right)^j \pmod p$$

构造：

$$c_i = \sum_{k=0}^{n-1} y_k \left(g_n^{-1}\right)^{ki} \pmod p$$

代入：

$$c_i = \sum_{k=0}^{n-1} \left(\sum_{j=0}^{n-1} a_j g_n^{kj} \right) g_n^{-ki} \pmod p$$

交换求和顺序：

$$c_i = \sum_{j=0}^{n-1} a_j \left(\sum_{k=0}^{n-1} \left(g_n^{j-i}\right)^k \right) \pmod p$$

对括号里面的东西用求和引理，当仅当$j = i$等于 n，否则等于 0。 于是：

$$c_i \equiv a_i \cdot n \pmod p$$

那么：

$$a_i \equiv c_i \cdot n^{-1} \pmod p$$

于是对做完点乘的点值做一次反向 NTT ，然后乘上 n 的逆元即可。

6 Trick 与杂项

6.1 数论/Trick/数学相关 trick

我们在处理某些计数问题时 总是有一下思路 1.定 1 移 1 考虑某个前缀对 r 的贡献 2.考虑某个点对区间的贡献 3.移项，解耦变量后考虑区间内相同性质点的贡献

数数题 最重要的思想是按什么东西进行分类，是好做的，而且能做到不重不漏

考虑排序是否对答案无影响

考虑算[l,r]中被 p 整除的数，一个好的实现是先算[1,r]中被 p 整除的数，再减去[1,l-1]中被 p 整除的数。

常系数线性递推式可以用矩阵快速幂优化。注意$dp[i]=W[1]dp[i-1]+W[2]dp[i-2]+...+W[d]dp[i-d]$。构造的矩阵为第一行为系数矩阵，然后是从(2,1)开始的类似于单位矩阵的矩阵(d 行 d 列)。 可以把

$$(dp[i-1],dp[i-2],...,dp[i-d])$$

看作一个列向量，然后乘上构造的矩阵，就可以得到

$$(dp[i],dp[i-1],...,dp[i-d+1])$$

。 有些时候能把状态压缩成一个等价类，缩减矩阵的大小

ntt 如果有多次 intt，有可能可以不用先 intt 然后答案相加，而是直接在 ntt 后把点值加起来，最后做一次 intt 就行

质数间距是小的，根据 x 附近的质数密度是 1/lnx，那么 x 附近的质数间距是 lnx

筛积性函数的时候 这个积性函数在质数和质数幂时候的值是好算的话 这样他的值是好求的。

维护多项式，我们如果想要动态加点/点集不变，多次查询，这样的式子，我们可以把拉插式子中无关 x 的项提出来，这样可以快速维护

$$P(x)=\sum_{i=1}^{n}y_i\prod_{j\neq i}\frac{x-x_j}{x_i-x_j}$$

令： $\ell(x)=\prod_{i=1}^{n}(x-x_i)$ $w_i=\frac{1}{\prod_{j\neq i}(x_i-x_j)}$ 那么

$$P(x)=\ell(x)\sum_{i=1}^{n}\frac{y_i}{(x-x_i)\prod_{j\neq i}(x_i-x_j)}$$

$$P(x)=\ell(x)\sum_{i=1}^{n}\frac{w_iy_i}{x-x_i}$$

多项式的前缀和(对 x 而言)是更高一次的多项式->积分 多项式的一阶差分(对 x 而言)是低一次的多项式->微分

处理逆元不存在的方法总列：考虑前后缀积处理 考虑处理乘 0 的个数 阶乘考虑列让德公式 考虑卢卡斯定理 考虑特殊性质，如 k 是 9 的倍数，要求 k/9 mod mod 那么可以先求 k%9mod 然后/9 然后 %mod

快速算(a+bx)^n 这样的事情可以考虑多项式启发式卷积,就是用过线段树向上合并的过程来描述，不过实现上队列就能做到。

有个事情就是 对 %n 来说 如果+dx gcd(dx,n)=1 那其实是遍历所有余数的 那 gcd=2 的话 那其实就变成两个不交同余类了

6.2 数论/数论笔记部分/数论笔记(杂项)

6.2.1 勒让德公式

6.2.1.1 勒让德公式是什么

勒让德公式用来算质数 p 在阶乘 $n!$ 的质因子分解中的指数。 公式是：

$$v_p(n!)=\sum_{k=1}^{\infty}\left\lfloor\frac{n}{p^k}\right\rfloor,$$

其中 $v_p(n!)$ 表示 p 在 $n!$ 里出现的次数（指数）。

6.2.1.2 为什么成立

考虑 $n!=1\cdot2\cdot3\cdots n$。

- 每隔 p 个数里有一个能被 p 整除，所以至少有 $\lfloor n/p\rfloor$ 个因子 p；
- 每隔 p^2 个数里有一个能被 p^2 整除，它会额外贡献一个 p，所以再加 $\lfloor n/p^2\rfloor$；
- 每隔 p^3 个数里有一个能被 p^3 整除，它会再额外贡献一个 p…… 如此类推，直到 $p^k>n$ 为止。

所以总和就是上面的式子。

6.2.1.3 一个例子

算 $v_2(10!)$：

$$\lfloor10/2\rfloor+\lfloor10/4\rfloor+\lfloor10/8\rfloor=5+2+1=8.$$

验证： $10! = 3628800 = 2^8 \cdot 3^4 \cdot 5^2 \cdot 7$.

6.2.1.4 在这题里的作用
我们要计算：

$$C_n = \frac{1}{n+1}\binom{2n}{n}.$$

其中涉及到阶乘： $\binom{2n}{n} = \frac{(2n)!}{(n!)^2}$。

为了在任意模数 p 下算出结果（尤其是 p 不是质数时），不能用逆元，要直接做质因子分解。于是对每个质数 q，用勒让德公式求它在 $(2n)!$、$n!$ 中的指数差，得到 q 在组合数中的幂次。

再减去 $n+1$ 的质因子指数，就是卡特兰数的质因子分解。最后用快速幂拼起来，就得到答案 $\bmod p$。

6.3 数论/数论笔记部分/数学笔记(矩阵半环)

6.3.1 半环

一个代数系统 $(R, \oplus, \otimes)$ 被称为半环，满足以下四个条件：

1. 加法构成交换幺半群，即：
   - 结合律：$(a \oplus b) \oplus c = a \oplus (b \oplus c)$
   - 交换律：$a \oplus b = b \oplus a$
   - 存在加法幺元（零元）$\bar{0}$：$a \oplus \bar{0} = \bar{0} \oplus a = a$
2. 乘法构成幺半群，即：
   - 结合律：$(a \otimes b) \otimes c = a \otimes (b \otimes c)$
   - 存在乘法幺元（单位元）$\bar{1}$：$a \otimes \bar{1} = \bar{1} \otimes a = a$
3. 乘法对加法存在分配律，即：
   - 左分配律：$a \otimes (b \oplus c) = (a \otimes b) \oplus (a \otimes c)$
   - 右分配律：$(a \oplus b) \otimes c = (a \otimes c) \oplus (b \otimes c)$
4. 加法幺元 $\bar{0}$ 对于乘法必须是吸收元：$a \otimes \bar{0} = \bar{0} \otimes a = \bar{0}$

6.3.2 常见矩阵半环

矩阵的单位阵，类比普通矩阵的单位阵，就是把 $0, 1$ 替换成 $\bar{0}, \bar{1}$。

如果要描述 $\leq k$，可以给矩阵增加一个维度表示汇点，然后给汇点加上自环。

6.3.2.1 1. 普通矩阵乘法

定义：

$$C_{i,j} = \sum_k (A_{i,k} \times B_{k,j})$$

- $\oplus$（广义加）：$+$（普通加法）
- $\otimes$（广义乘）：$\times$（普通乘法）
- 幺元：$\bar{0} = 0$，$\bar{1} = 1$
- 物理意义（用处）：
  1. 邻接矩阵经过矩阵乘法 A^k 后，矩阵 (i, j) 位置的值就代表：从节点 i 出发，恰好经过 k 条边到达节点 j 的路径总方案数。原因是外层的加号表示遍历某个中转点，而内层的乘法相当于乘法原理，而走的长度则正好是矩阵的幂次和。
  2. 加速某些常系数线性递推式。

此处，邻接矩阵 A 中的元素 $A_{i,j}$ 表示节点 i 到节点 j 的直接连边数量。

6.3.2.2 2.1. min-plus 矩阵乘法

定义：

$$C_{i,j} = \min_k (A_{i,k} + B_{k,j})$$

- $\oplus$（广义加）：$\min$（最小值）
- $\otimes$（广义乘）：$+$（普通加法）
- 幺元：$\bar{0} = +\infty$，$\bar{1} = 0$
- 物理意义（用处）：定长最短路问题，将图的邻接矩阵 A 进行 k 次广义自乘，得到的 A^k 中，(i, j) 位置的值代表“从 i 到 j 恰好走 k 条边的最短路径长度”。原因同样是遍历所有中转点，取 min，内层加法相当于拼接，而走的长度则正好是矩阵的幂次和。

此处，邻接矩阵 A 中的元素 $A_{i,j}$ 表示节点 i 到节点 j 的所有边权的 min，无边设为 $+\infty$。

6.3.2.3 2.2. max-plus 矩阵乘法

定义：

$$C_{i,j}=\max_k(A_{i,k}+B_{k,j})$$

- $\oplus$（广义加）：$\max$（最大值）
- $\otimes$（广义乘）：$+$（普通加法）
- 幺元：$\bar{0}=-\infty$，$\bar{1}=0$
- 物理意义（用处）：
  1. 定长最长路，不再赘述
  2. 某些 dp 转移

此处，邻接矩阵 A 中的元素 $A_{i,j}$ 表示节点 i 到节点 j 的所有边权的 max，无边设为 $-\infty$；单位矩阵的对角线为 0。

6.3.2.4 3.1. max-min 矩阵乘法
定义：

$$C_{i,j}=\max_k(\min(A_{i,k},B_{k,j}))$$

- $\oplus$（广义加）：$\max$（最大值）
- $\otimes$（广义乘）：$\min$（最小值）
- 幺元：$\bar{0}=-\infty$，$\bar{1}=+\infty$
- 物理意义（用处）：定长最大瓶颈路：A^k 的 (i,j) 就代表从 i 到 j 恰好经过 k 条边时，所有路径中对于每个路径最小边权的最大值。考虑一个现实场景：火车运载的场景，显然你要选择某些有短板的路径中，短板最长的那个。

此处，邻接矩阵 A 中的元素 $A_{i,j}$ 表示节点 i 到节点 j 的所有边权的 max，无边设为 $-\infty$。

6.3.2.5 3.2. min-max 矩阵乘法
定义：

$$C_{i,j}=\min_k(\max(A_{i,k},B_{k,j}))$$

- $\oplus$（广义加）：$\min$（最小值）
- $\otimes$（广义乘）：$\max$（最大值）
- 幺元：$\bar{0}=+\infty$，$\bar{1}=-\infty$
- 物理意义（用处）：定长最小瓶颈路：A^k 的 (i,j) 就代表从 i 到 j 恰好经过 k 条边时，所有路径中对于每个路径最大边权的最小值。考虑一个现实场景：网络传递的场景，边权代表丢包率，我们希望最卡的路径丢包率最低。

此处，邻接矩阵 A 中的元素 $A_{i,j}$ 表示节点 i 到节点 j 的所有边权的 min，无边设为 $+\infty$。

6.3.2.6 4. 布尔半环
定义：

$$C_{i,j}=\vee_k\left(A_{i,k}\wedge B_{k,j}\right)$$

- $\oplus$（广义加）：$\vee$（或）
- $\otimes$（广义乘）：$\wedge$（与）
- 幺元：$\bar{0}=0$（逻辑 0），$\bar{1}=1$（逻辑 1）
- 物理意义（用处）：布尔矩阵乘法，A^k 的 (i,j) 就代表从 i 到 j 恰好经过 k 条边时，所有路径中是否存在一条路径是通的。

可以用来判断是否存在长度 k 的环。这个东西可以用 bitset 优化一下。

此处，邻接矩阵 A 中的元素 $A_{i,j}$ 表示节点 i 到节点 j 是否存在一条边，无边设为 0（逻辑 0）。

6.3.2.7 5. 概率半环
定义：

$$C_{i,j}=\max_k(A_{i,k}\times B_{k,j})$$

- $\oplus$（广义加）：$\max$（最大值）
- $\otimes$（广义乘）：$\times$（普通乘法）
- 幺元：$\bar{0}=0$，$\bar{1}=1$
- 物理意义（用处）：概率矩阵乘法，A^k 的 (i,j) 就代表从 i 到 j 恰好经过 k 条边时，所有路径中存在一条路径的最大概率。

考虑浮点数乘法的精度问题，我们做如下映射：$f(x)=-\log(x)$，$\prod p_i\Longrightarrow\sum-\log(p_i)$，$\max(\prod p_i)\Longrightarrow\min(\sum-\log(p_i))$，这样我们就可以用 min-plus 矩阵乘法来处理概率矩阵乘法了。还原就 exp 回去就行。

此处，邻接矩阵 A 中的元素 $A_{i,j}$ 表示节点 i 到节点 j 的所有边权的概率，无边设为 0（概率 0）。

6.3.2.8 6. 异或-与环
定义：

$$C_{i,j} = \oplus_k \left(A_{i,k} \wedge B_{k,j} \right)$$

- $\oplus$（广义加）：$\oplus$（异或）
- $\otimes$（广义乘）：$\wedge$（与）
- 幺元：$\bar{0} = 0$，$\bar{1} = \text{全 } 1 \text{ 掩码}$
- 物理意义（用处）：异或-与矩阵乘法，A^k 的 $(i,j) = 1$ 就代表从 i 到 j 恰好经过 k 条边时，所有路径数为奇数。

此处，邻接矩阵 A 中的元素 $A_{i,j}$ 表示节点 i 到节点 j 是否存在一条边，无边设为 0（逻辑 0）。

6.3.2.9 7. 最短路计数半环
考虑：

- 元素为 (d, c)，d 为距离，c 为方案数。
- $\oplus$（广义加）：代表在多条路径中择优。

当两条路径集合 (d_1, c_1) 和 (d_2, c_2) 合并时：

- 如果 $d_1 < d_2$，选前者：结果为 (d_1, c_1)
- 如果 $d_1 > d_2$，选后者：结果为 (d_2, c_2)
- 如果 $d_1 = d_2$，产生平局，方案数相加：结果为 $(d_1, c_1 + c_2)$

$\otimes$（广义乘）：代表路径的拼接。

将第一段路径 (d_1, c_1) 和第二段路径 (d_2, c_2) 拼接起来：

- 距离直接相加：$d_{\text{new}} = d_1 + d_2$
- 方案数依据乘法原理相乘：$c_{\text{new}} = c_1 \times c_2$
- 结果为 $(d_1 + d_2, c_1 \times c_2)$

幺元：

- 加法幺元 $\bar{0} = (+\infty, 0)$（距离无穷大，方案数为 0）
- 乘法幺元 $\bar{1} = (0, 1)$（距离为 0 的原地踏步，方案数为 1）

物理意义（用处）：

A^k 的 (i, j) 就代表从 i 到 j 恰好经过 k 条边时，所有路径的最短路长度，以及满足该长度的路径条数。

此处，邻接矩阵 A 中的元素 $A_{i,j}$ 表示节点 i 到节点 j 中所有重边按广义加合并，无边设为加法幺元。

作业：https://codeforces.com/gym/102644